

Architecture & Implementation Guide

Version 1.0 · React 19 + Vite 8 progressive web app · No backend, no database

Who this is for. A React developer who has never seen this codebase. After reading you should be able to explain how the app starts, where data lives, how sign-in and sync work, how every feature is built, why each choice was made, and how to rebuild it step by step.

Repository: github.com/ArunKathir98/PocketZen · Live app: pocketzen.arunkumar98kak.workers.dev

How to read this guide

The guide goes from the big picture to the details, then finishes with a step-by-step build log and a maintenance playbook. If you are short on time, read **chapters 1, 3, 5 and 17**; they give you the mental model.

1 The product and its constraints
2 Tech stack and packages
3 Architecture at a glance
4 Project map (every file)
5 Startup and routing flow
6 State management and handlers
7 Persistence and Drive sync
8 Authentication and security
9 Feature modules
10 Shared groups
11 The UI layer
12 PWA, offline and notifications
13 Build, configuration, deployment
14 Testing approach
15 Step-by-step implementation
16 Decisions, advantages, trade-offs
17 Maintenance playbook
A Storage keys · Google calls · handlers · glossary

Conventions used in this document

> **NOTE**
> Blue boxes add context you might otherwise miss.

> **WHY THIS WAY**
> Purple boxes explain **why** a design choice was made, including the alternatives that were rejected.

> **ADVANTAGE**
> Green boxes call out the concrete advantage you get from a choice.

> **WATCH OUT**
> Amber boxes mark traps, limits and things that surprised us while building.

File paths are relative to the repository root. `sub` means a Google account's unique id (or the literal `"local"` for the no-account mode). Money amounts are plain numbers in the user's currency, except inside shared groups, where they are converted to whole minor units (cents) for exact arithmetic.

The five-minute summary

1. PocketZen is a **static single-page app**. There is no PocketZen server. Cloudflare only serves the built files.
2. All user data lives in one JSON document. It is kept in **IndexedDB** on the device and, for Google users, mirrored to a hidden file in the user's own **Google Drive** (`drive.appdata`).

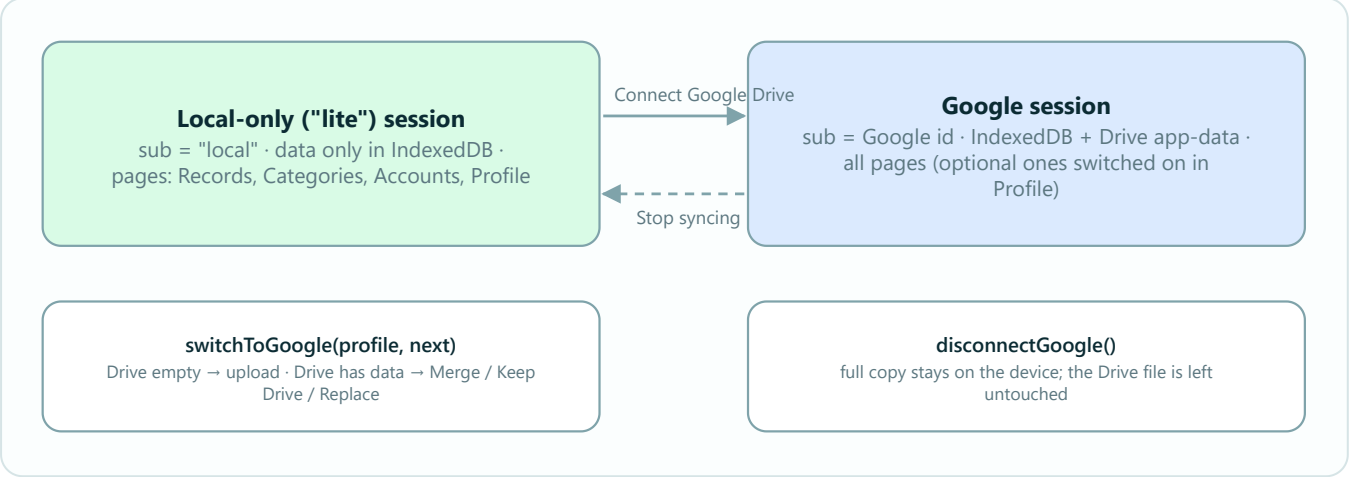
3. React state is a single **Context** (`store.jsx`). Every change goes through one function, `commit()`, which updates the UI, saves locally and schedules a Drive sync.
4. Sign-in uses **Google Identity Services (token model)**: the browser gets a one-hour access token and calls the Drive REST API directly.
5. People without a Google account use a **lite mode** (Records, Categories, Accounts) that never leaves the device.
6. Calculation-heavy features (loan interest, group settle-up, balances) are **pure functions** with no React inside, so they are easy to test.
7. Shared groups are **files in the admin's Drive** that anyone with a link can view. Only the admin edits.

1 · The product and its constraints

1.1 What PocketZen does

Page	What the user can do	Available to
Records	Add income, expense and transfers; daily / weekly / monthly views with totals; edit or delete	Everyone
Categories	Manage income and expense categories; tap one to see all its transactions by period	Everyone
Accounts	Bank, wallet, cash, investment accounts with live balances	Everyone
Budget	Monthly limit per expense category; remaining balance shown while adding an expense	Google users
EMIs	Ongoing instalment loans: total, months, pending months, payments, schedule	Google users (optional page)
Loans & people	Money lent or borrowed with fixed, simple, compound or manual interest; statements	Google users (optional page)
Groups	Split expenses with friends; settle-up suggestions; PDF summary; view-only share link	Google users (optional page)
Reminders	Recurring bill reminders and a daily "add today's expenses" nudge, with sound	Google users (optional page)
Profile	Theme, currency, PIN, notifications, backup and restore, share, log out	Everyone

1.2 Two ways to use the app



	Google session	Local ("lite") session
Identity	`session = { sub, email, name, picture, mode:'google' }`	`session = { sub:'local', mode:'local' }`
Data location	IndexedDB + Drive app-data file	IndexedDB only
Pages	All; optional ones are toggled in Profile → Pages	Records, Categories, Accounts, Profile
Backup	Automatic (Drive)	Backup file, optional password; reminder after 14 days
Forgot PIN	Re-verify with Google	One-time recovery code

1.3 The constraints that shaped every decision

Constraint	Consequence in the design
"Your data stays yours" — no server holding financial data	Data lives in the user's own Google Drive or on their device. The app is a static site. Even shared groups are files in the admin's Drive.
Free to run	Cloudflare Workers static assets (free), Google Drive (user's own quota), no database, no paid services.
Works like a phone app	PWA: manifest, service worker, install prompt, offline cache, responsive layout from phone to monitor.
Some users won't use Google	Local-only lite mode with encrypted backups and a clean upgrade path to Drive.
No backend means no refresh tokens, no push server, no secrets	Short-lived browser tokens cached carefully; reminders only fire while the app can run; only public identifiers (client id, API key) ship in the bundle.
Future maintainers are mostly front-end developers	Plain React + Context, hand-written CSS, almost no dependencies, business logic isolated in pure functions.

WHY THIS WAY

The single most important decision is **"no backend"**. It removes servers, accounts, databases, GDPR duties and running costs, and it makes the privacy promise literally true. The price is that everything a server normally does (sync, conflict handling, scheduling, sharing) has to be solved in the browser. Chapters 7, 8, 10 and 12 show how.

2 · Tech stack and packages

The dependency list is deliberately tiny: **three runtime packages**. Everything else is browser platform APIs.

2.1 Runtime dependencies (shipped to users)

Package	Version	Role	Why this and not something else
react	^19.3	Component model, hooks, Context, `StrictMode`	Familiar to the target maintainers. Hooks + Context are enough for this app's state.
react-dom	^19.3	Renders React in the browser (`createRoot`)	Required by React.
lucide-react	^1.61	Icons as small SVG components (tree-shaken per import)	Consistent outline style, no icon font download, only the icons used end up in the bundle.

2.2 Development and build dependencies

Package	Version	Role	Why
vite	^8.3	Dev server (instant start, hot reload) and production bundler	Fast, almost zero config for React, first-class env variables (`VITE_*`).
@vitejs/plugin-react	^6.1	JSX transform and Fast Refresh	Official React plugin for Vite.
vite-plugin-pwa	^2.0	Generates `manifest.webmanifest` and a Workbox service worker that precaches the app	Gives an installable, offline-capable PWA without hand-writing a service worker.
wrangler	^4.152	Cloudflare's CLI; `npx wrangler deploy` publishes `dist/` as static assets	Pinned in the repo so Cloudflare's Git build uses a known version.
sharp	^0.35	Build-time image processing (`npm run icons` , poster JPEG)	Turns the white-background logo into a transparent PNG and generates every PWA icon size.
qrcode	^1.5	Draws the QR code on the share poster	Tiny, outputs SVG.
playwright-core	^1.64	Drives a headless Chrome/Edge to render the poster and this PDF	Reuses the browser already installed, so no big download.

2.3 Things we deliberately did *not* add

Not used	What we did instead
React Router	The app has one URL. The active page is a `tab` state in `App.jsx`, remembered in `localStorage`. Fewer moving parts and no server rewrite rules.
Redux / Zustand	One Context (`store.jsx`) holding one JSON document, plus refs for the latest value. Easy to reason about and trivially serialisable to Drive.
Tailwind / a UI kit	One hand-written `styles.css` with CSS custom properties (design tokens), light/dark themes and a few reusable classes.
date-fns / moment	~80 lines in `util.js`; dates are stored as `'YYYY-MM-DD'` strings, which sort correctly and avoid time-zone bugs.
A crypto library	The Web Crypto API (PBKDF2, AES-GCM, SHA-256) built into every browser.
A PDF library (jsPDF…)	Pages are drawn on a canvas and packed into a PDF by a 40-line writer, so any language and the ₹ symbol render correctly (standard PDF fonts can't).
The Google API client library (gapi)	Plain `fetch` calls to the Drive REST API; only Google Identity Services is loaded as a script.
A backend framework or database	None. See chapter 1.

ADVANTAGE

Small dependency surface = small bundle (about 440 KB of JavaScript, roughly 130 KB compressed, for the whole app), few security advisories to track, and nothing that can break on an upgrade except React and Vite.

2.4 Browser platform APIs the app relies on

API	Used for	File
IndexedDB	Main on-device database (user data, cached groups, outboxes)	`lib/idb.js`
localStorage	Small flags: session, token, dirty / synced markers, tab, theme	various
Web Crypto	PIN and recovery-code hashing (PBKDF2), encrypted backups (AES-GCM)	`pin.js`, `backup.js`
Google Identity Services	OAuth access tokens (script loaded in `index.html`)	`google.js`
Drive REST API v3 (fetch)	Reading and writing the data file and group files	`google.js`
Service Worker + Notifications	Offline cache, system notifications, click handling	`public/sw-notify.js`, `notify.js`

API	Used for	File
Web Audio + Vibration	Reminder chime and haptics while the app is open	`notify.js`
Web Share + Clipboard	Share app, backups, PDFs and invites; copy text	several components
Canvas 2D	Draws the group summary pages for the PDF	`groupPdf.js`
Intl.NumberFormat / toLocale…	Currency and date formatting in the user's locale	`util.js`, `groupState.js`
`beforeinstallprompt`, **display-mode**	Custom "Install app" banner	`install.js`

3 · Architecture at a glance

3.1 The seven layers

Think of the code as seven layers. Each layer mostly talks to the one directly below it, which is what keeps the app understandable.

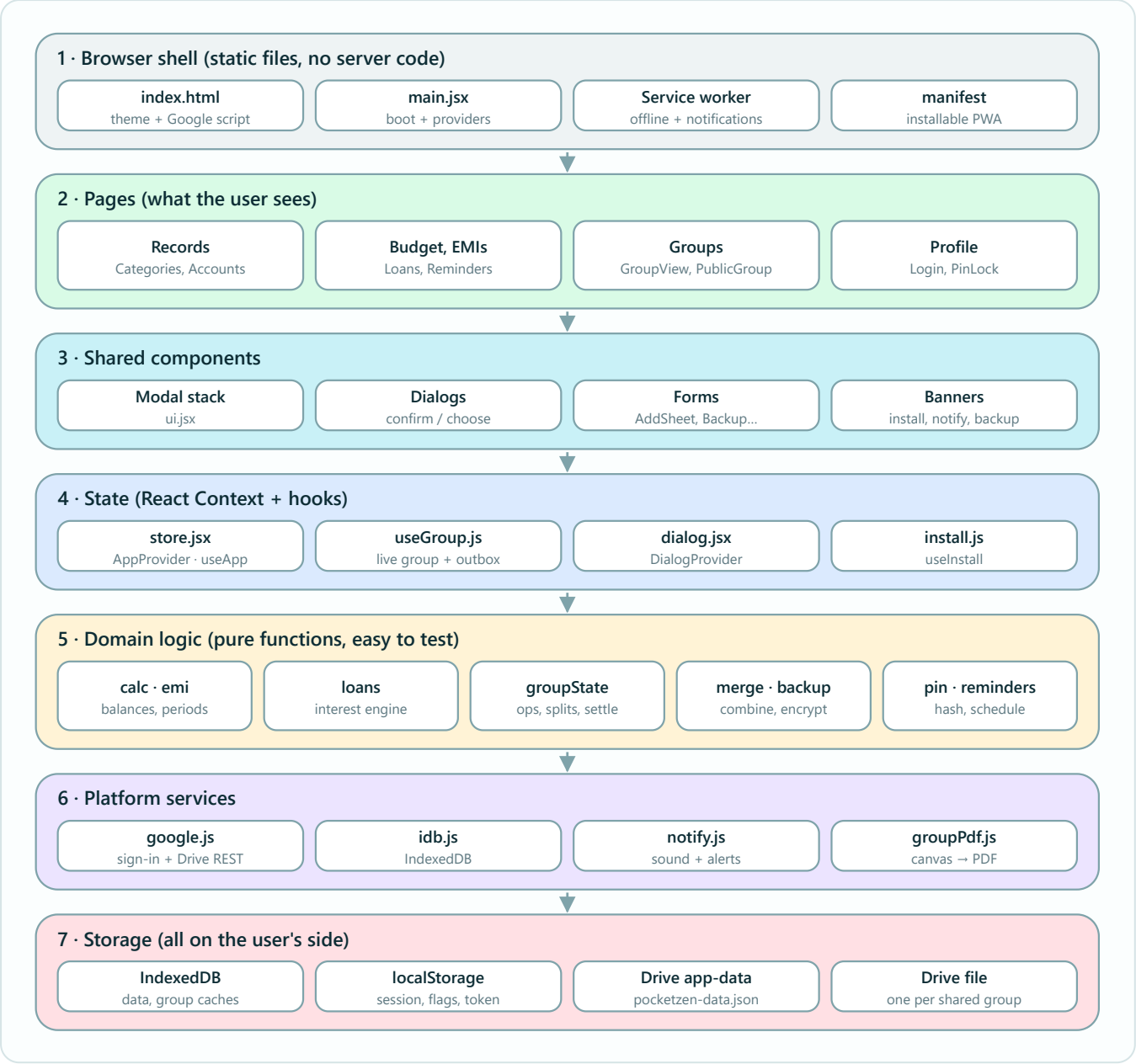


Figure 3.1 — Layers of PocketZen. Most pages only call the store or a pure function. The group screens (Groups, GroupView) and CloudCard also call google.js and the IndexedDB helper directly, because group data is separate from the personal document.

3.2 One document, three places

All personal data is one JSON object (the "document"). It is kept in up to three places, and understanding which is which explains most of the code:

Place	Holds	Written by
React state (`data`)	The live copy the UI renders. Mirrored in a ref (`dataRef`) so async code always sees the latest value.	`commit()`
IndexedDB `data:<sub>`	The durable on-device copy. The app starts from it, so it works offline and loads instantly.	`persistLocal()` after every commit
Drive `pocketzen-data.json`	The cloud copy (Google users only), in the hidden app-data folder. Lets several devices share data.	`push()` / read by `pull()`

> **WHY THIS WAY**
>
> **Local first.** The UI never waits for the network. A change shows immediately, is saved to IndexedDB in milliseconds, and Drive catches up in the background. If Google is unreachable the app keeps working and syncs later.

3.3 Request and data flow in one picture

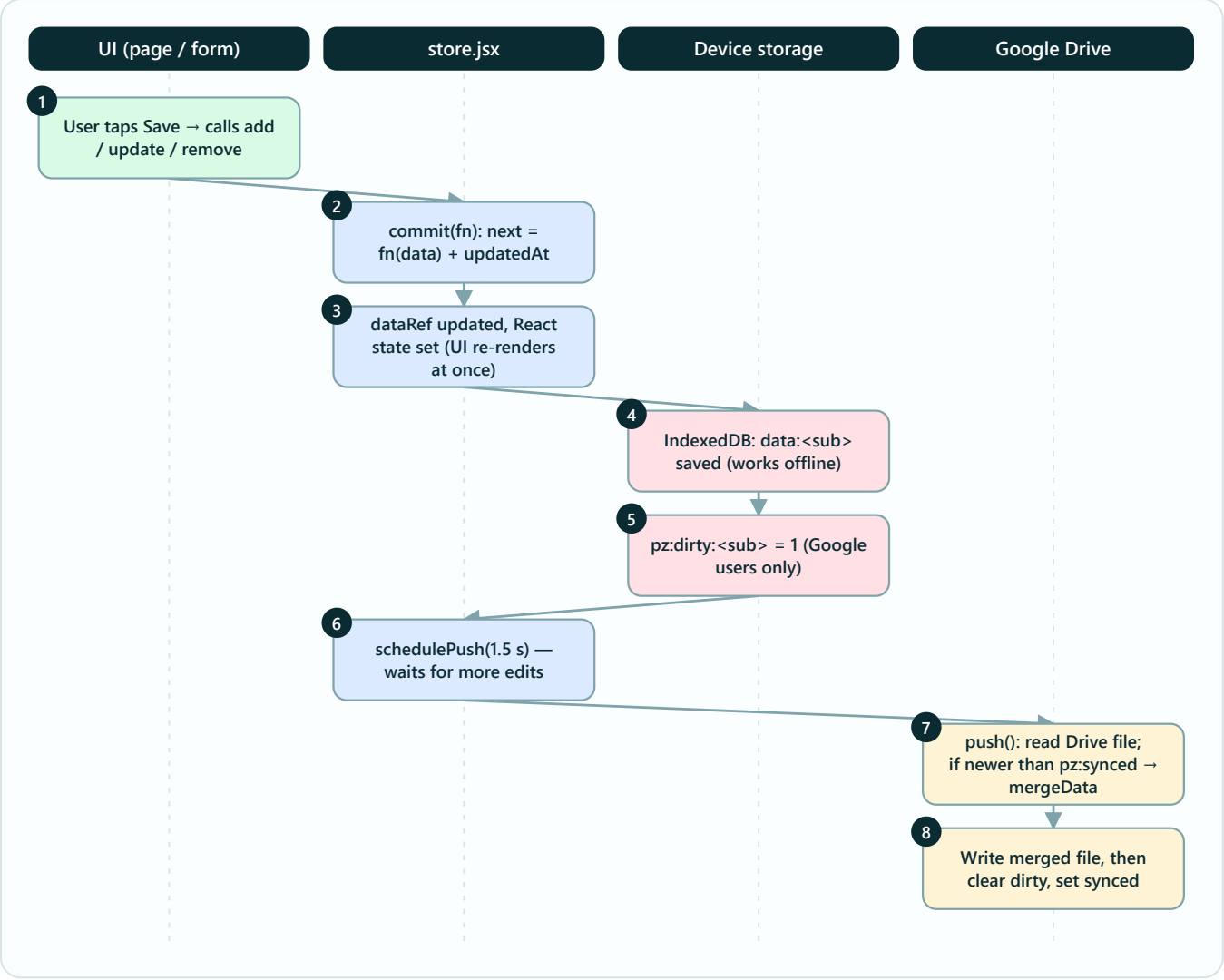


Figure 3.2 — What happens when the user saves a record (details in chapters 6 and 7).

3.4 Separation of concerns

Kind of code	Rule	Examples
Pure domain logic	No React, no DOM, no storage. Input → output. Unit-testable in plain Node.	`calc.js, loans.js, emi.js, groupState.js, merge.js, backup.js, pin.js`
Platform services	Talk to the browser or Google. May use `window`, `fetch`, IndexedDB.	`google.js, idb.js, notify.js, install.js, groupPdf.js`
State	React Context/hooks that glue services and logic together.	`store.jsx, useGroup.js, dialog.jsx`
UI	Render and collect input. No business rules; call the store or a pure function.	`pages/*, components/*`

> **ADVANTAGE**
> When a bug is about *numbers* (interest, balances, splits) you only ever open a pure module. When it is about *sync or sign-in* you open `store.jsx` / `google.js`. When it is about *looks* you open a page and `styles.css`.

3.5 Two kinds of shared state

- **Personal data** (accounts, records, budgets, EMIs, loans, reminders…): one document per user, managed by `store.jsx`, synced as a whole file.
- **Group data**: one document per group, a separate file in the admin's Drive, managed by `useGroup.js` with its own cache and outbox. They are kept separate because they have different owners, audiences and sync rules (chapter 10).

4 · Project map (every file)

```
# repository root
index.html            # theme-before-paint script, loads Google Identity Services, mounts #root
vite.config.js        # React plugin + PWA plugin (manifest, Workbox, importScripts)
wrangler.jsonc        # Cloudflare: serve dist/ as static assets (+ retire an old Durable Object)
worker/index.js       # 3-line Worker that hands every request to the static assets
.env.example          # VITE_GOOGLE_CLIENT_ID, VITE_GOOGLE_API_KEY
public/               # icons, sw-notify.js, home.html, privacy.html, share-poster.jpg
scripts/              # make-icons.mjs, poster/*, docs/* (this PDF)
src/
  main.jsx            # boot: capture invite link, install listener, audio unlock, SW, providers
  App.jsx             # the router: splash → PIN → public group → login → app shell + tabs
  config.js           # feedback form URL, GROUP_SHARING switch
  styles.css          # design tokens, light/dark, layout, every component style
  lib/                # state, services and pure logic (below)
  pages/              # one file per screen
  components/         # reusable pieces
  assets/             # logo.png (transparent), logo-source.jpg
```

4.1 `src/lib` — state, services, logic

File	Kind	Responsibility
store.jsx	State	`AppProvider` and `useApp()`: session, data, lock, sync status; `commit`, `push`, `pull`, login, logout, mode switching.
defaults.js	Data	`emptyData()` shape, default categories/accounts/settings, account types, currencies, colours, emoji list.
idb.js	Service	Promise key-value store on IndexedDB with a localStorage fallback.
google.js	Service	Token handling (cache, scopes, renewal), profile, Drive app-data load/save, group file create/share/read/write/delete, public read.
merge.js	Logic	`mergeData(base, inc)` — combine two documents without duplicates; `hasUserData`.
calc.js	Logic	Account balances, period ranges and labels, totals, per-category monthly spend, day grouping.
emi.js	Logic	EMI statistics (paid, pending, remaining, next due) and upcoming schedule.
loans.js	Logic	Loan interest engine (`loanStats`), interest-first split, person totals, statement text.
groupState.js	Logic	Group data model: ops + last-write-wins merge, validation limits, equal split, balances, settle-up algorithm, money formatting.
useGroup.js	State	Live view of one group: cache, polling, outbox, admin/viewer roles.
groupPdf.js	Service	Draws summary pages on canvas and writes a PDF; share-or-download helper.

File	Kind	Responsibility
backup.js	Logic	Build, encrypt, parse and save backup files.
pin.js	Logic	PIN hashing/verification and the one-time recovery code.
reminders.js	Logic	Recurrence maths, `checkDue()` for bills, loans and the daily nudge.
notify.js	Service	Chime, vibration, system notifications, `fireReminder()`.
install.js	Service	Captures `beforeinstallprompt`; `useInstall()` hook.
pendingJoin.js	Service	Remembers a `?view=` group link across sign-in.
util.js	Helpers	ids, date helpers, money formatting, download helper.

4.2 `src/pages` — screens

File	Screen
Login.jsx	Welcome + "Continue with Google" / "Continue without an account".
PinLock.jsx	PIN pad and both "Forgot PIN" flows (Google re-verify, local recovery code).
Records.jsx	Totals hero, day/week/month switch, grouped list, add button.
Categories.jsx	Income/expense tabs, add/edit/delete, opens category detail.
Accounts.jsx	Account cards with computed balances.
Budget.jsx	Per-category monthly limits with progress.
Emi.jsx	Ongoing/completed EMIs, pay flow, detail view with schedule and history.
Loans.jsx	People list, person detail, loan detail, payment/terms forms, statements.
Groups.jsx	My groups, create, join (view link).
GroupView.jsx	One group: expenses, payments, people, events, settle-up, summary/PDF, invite. Read-only for viewers.
PublicGroup.jsx	Wrapper that shows a group to someone with no session.
Reminders.jsx	Bill reminders list and form.
User.jsx	Profile: backup & sync, theme, preferences, notifications, pages, security, data, share, log out.

4.3 `src/components` — reusable pieces

File	Purpose
ui.jsx	`Modal` (stack-aware), `Toggle`, `Empty`, `Logo`, Google "G" icon.
dialog.jsx	`DialogProvider` / `useDialog()` — promise-based `confirm`, `alert`, `choose` that replace browser pop-ups.

File	Purpose
AddSheet.jsx	Add/edit a record (income, expense, transfer) incl. remaining-budget hint.
CategoryDetail.jsx	All transactions of one category with period filter and budget bar.
PinPad.jsx	Keypad + dots, keyboard support, shake on wrong PIN.
CloudCard.jsx	Local ⇄ Google switching UI (connect, merge choices, disconnect).
BackupModal.jsx / RecoveryModal.jsx	Create (optionally encrypted) backups; show the recovery code once.
NotificationsCard.jsx	Permission status, sound toggle, test buttons.
ShareCard.jsx	Share the app (message + poster), WhatsApp, copy, feedback form.
Banners.jsx	`InstallBanner`, `NotifyBanner`, `BackupBanner`.
LocalWelcome.jsx	One-time tip shown to people who start without an account.

5 · Startup and routing flow

There is no router library. "Routing" is a chain of early returns in `App.jsx`. Read this chapter once and you know what the user sees in every situation.

5.1 What runs before React

1. **`index.html`**
   An inline script reads `localStorage['pz:theme']` and sets `<html data-theme="light|dark">` *before* first paint, so there is no white flash in dark mode. Then it loads Google Identity Services (`accounts.google.com/gsi/client` , `async defer`) and the module `/src/main.jsx` .

2. **`main.jsx`**
   In order: `capturePendingJoin()` (if the URL has `?view=<id>` store the id in `pz:pendingJoin` and clean the address bar); import `install.js` (starts listening for `beforeinstallprompt` before React mounts, because the event can fire early); register one-time `pointerdown` / `keydown` listeners that unlock Web Audio (browsers only allow sound after a gesture); `registerSW({ immediate: true })` ; finally render `<StrictMode><AppProvider><DialogProvider><App /></DialogProvider></AppProvider></StrictMode>` .

3. **`AppProvider` (store.jsx)**
   Reads `pz:session` synchronously. If there is a session it loads that user's document from IndexedDB (asynchronously; one-time migration from the old localStorage copy), sets `locked = !!data.pin` and finally `ready = true` . Without a session `ready` is immediately true.

5.2 The decision chain in `App.jsx`

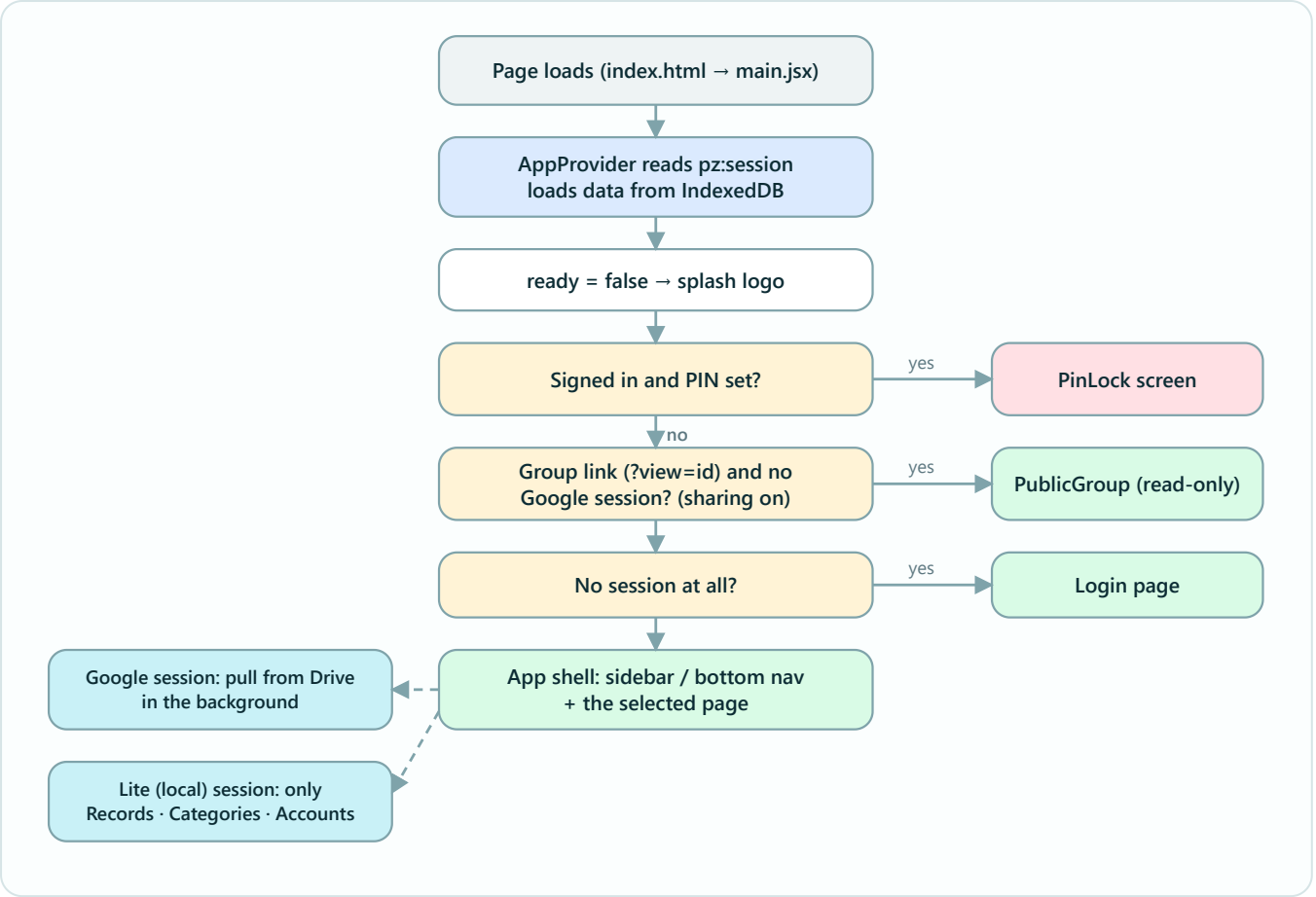


Figure 5.1 — Which screen is shown. Order matters: the PIN comes before everything else for signed-in users.

#	Condition	Result
1	`!ready`	Splash with the logo (data still loading from IndexedDB).
2	`session && locked`	`PinLock`.
3	group link pending, sharing enabled, and (`!session` or local mode)	`PublicGroup` — read-only group, no sign-in.
4	`!session`	`Login`.
5	otherwise	App shell: sidebar (≥ 820 px) or bottom bar (< 820 px) plus the page for the selected `tab`.

5.3 Which tabs exist

```
const TABS = [
  ['records',    'Records',    ListChecks,  Records],
  ['categories', 'Categories', LayoutGrid,  Categories],
  ['accounts',   'Accounts',   Landmark,    Accounts],
  ['budget',     'Budget',     PiggyBank,   Budget],
  ['emi',        'EMIs',       HandCoins,   Emi,       'showEmi'],
  ['groups',     'Groups',     Users,       Groups,    'showGroups'],
  ['loans',      'Loans',      Handshake,   Loans,     'showLoans'],
  ['reminders',  'Reminders',  BellRing,    Reminders, 'showReminders'],
  ['user',       'Profile',    CircleUserRound, User],
]
// lite (local) mode only gets these four:
const LITE_TABS = new Set(['records', 'categories', 'accounts', 'user'])
const tabs = TABS.filter(t => isLocal ? LITE_TABS.has(t[0]) : !t[4] || data.settings[t[4]])
```

The fifth array element is the settings flag that switches an optional page on. If the saved `tab` is no longer allowed (for example after disabling a page) the app falls back to Records.

5.4 Effects that live in `App.jsx`

- **Remember the tab** in `pz:tab`.
- **Invite link while signed in with Google**: enables the Groups page and jumps to it (the Join dialog then reads the pending id).
- **Reminder scheduler**: `checkDue(data)` immediately, every 30 s, and whenever the tab becomes visible again; fired reminders become toasts. Skipped for locked sessions and local mode.

> **WHY THIS WAY**
>
> **Why early returns instead of a router?** The app is a single screen with tabs. A router would add a dependency and URL/rewrite concerns for no benefit, and every "state of the world" (loading, locked, public, logged out) is a different *whole screen* anyway, which early returns express most clearly.

> **WATCH OUT**
>
> **StrictMode.** In development React runs effects twice. Every effect that starts something (loading data, polling) cleans up after itself (an `alive` flag or `clearInterval`), so the double run is harmless. Keep that discipline when adding effects.

6 · State management and handlers

6.1 The data document

`emptyData()` in `lib/defaults.js` defines the whole document. It is the only thing that is saved, synced, exported and merged.

```
{
  version: 1,
  updatedAt: 0,                // ms timestamp of the last change — drives sync decisions
  accounts:     [],            // { id, name, type, balance (opening), color, note }
  categories:   [],            // { id, type:'income'|'expense', name, icon, color }
  transactions: [],            // { id, type:'income'|'expense'|'transfer', amount, accountId,
toAccountId,
                               //   categoryId, note, date:'YYYY-MM-DD', time:'HH:mm' }
  budgets:      [],            // { id, categoryId, amount, period:'monthly' }
  reminders:    [],            // { id, title, amount, dueAt, repeat, until, doneCount }
  emis:         [],            // { id, name, lender, totalAmount, totalMonths, emiAmount, startDate,
paidBefore, payments[], accountId }
  people:       [],            // { id, name, note }
  loans:        [],            // { id, personId, direction:'lent'|'borrowed', title, startDate,
interest{…}, dueDate, remind, note, events[] }
  groups:       [],            // groups I belong to: { id (Drive file id), name,
role:'admin'|'viewer', me, joinedAt }
  settings: { currency, dailyReminder, dailyReminderTime, showEmi, showReminders,
              showLoans, showGroups, notifySound, lastBackupAt },
  pin: null                    // { salt, hash, recovery? } — never exported
}
```

> **NOTE**
>
> **Account balances are not stored.** They are computed on demand (`calc.accountBalance` = opening balance + income − expense ± transfers). Storing derived values would let them drift from the records; computing them cannot.

> **WHY THIS WAY**
>
> **One document, not many tables.** A personal finance dataset is small (a few thousand records). One JSON object is trivial to persist, export, import, merge and sync as a single Drive file. A normalised database would add complexity with no benefit at this size.

Every record has an `id` created by `uid()` (`crypto.randomUUID()`). Ids are what make merging two documents safe (chapter 7).

6.2 `store.jsx` — the provider

`AppProvider` owns this React state:

State	Meaning
`session`	Who is signed in (`pz:session`), or `null` . Mode is `'google'` or `'local'` .
`data`	The live document.
`ready`	False until the document has loaded from IndexedDB.
`locked`	True while the PIN screen must be shown.
`sync`	`idle | local | syncing | synced | offline | auth | error` — drives the status pill and the "Reconnect" banner.
`loggingIn` , `error`	Login button state and message.

and these refs: `dataRef` and `sessionRef` (always the latest value, safe to read inside async callbacks), `pushTimer` , `pushing` .

> **WHY THIS WAY**
>
> **Why refs next to state?** Async code (a push that awaits Drive) would otherwise read a stale `data` captured by its closure. `dataRef.current` is updated synchronously inside `commit` , so a sync that finishes seconds later still sees the user's newest edit.

6.3 The one write path: `commit()`

```
const commit = useCallback((fn) => {
  const s = sessionRef.current
  const next = { ...fn(dataRef.current), updatedAt: Date.now() }
  dataRef.current = next            // 1. latest value for async code
  setData(next)                     // 2. re-render now
  if (s) {
    persistLocal(s.sub, next)       // 3. IndexedDB  data:<sub>
    if (s.mode !== 'local') {
      localStorage.setItem(dirtyKey(s.sub), '1')   // 4. "Drive is behind"
      schedulePush()                // 5. debounce 1.5 s → push()
    }
  }
}, [...])
```

Everything the UI can change goes through it via the small API below, so there is exactly one place to look for "who changed the data".

6.4 The context API (`useApp()`)

Name	What it does
`add(coll, item)`	Appends `{ id: uid(), ...item }` to a collection; returns the new item (so callers can use its id immediately).
`update(coll, id, patch)`	Shallow-merges a patch into one item.

Name	What it does
`remove(coll, id)`	Deletes one item.
`setSettings(patch)`	Merges into `settings`.
`setPin(record)`	Stores or clears the PIN record.
`replaceAll(doc)` / `mergeIn(doc)`	Import a backup: replace everything (keeping the current PIN and last-backup time) or merge with `mergeData`.
`lock()` / `unlock()`	Show or leave the PIN screen.
`login()`	Google sign-in flow (chapter 8).
`loginLocal()`	Start a no-account session (`sub = 'local'`) and ask the browser for persistent storage.
`reconnect()`	User-triggered token renewal (from the banner), then `pull()`.
`syncNow()`	Alias of `pull()` (the refresh button in Profile).
`logout()`	Flush pending edits to Drive, revoke the token, delete local data and the session. In local mode this means **erase everything**.
`switchToGoogle(profile, next)`	Local → Google: store `next` under the Google `sub`, mark dirty (so it uploads), remove the local copy.
`disconnectGoogle()`	Google → local: keep a full copy under `'local'`, leave the Drive file untouched.
`session, isLocal, ready, data, locked, sync, loggingIn, error`	Read-only values.

Usage in a page is deliberately boring:

```
const { data, add, update, remove } = useApp()
add('transactions', { type: 'expense', amount: 250, accountId, categoryId, date, time, note })
update('budgets', budget.id, { amount: 9000 })
```

6.5 Handlers and effects inside the provider

Handler / effect	Trigger	Behaviour
Startup load effect	mount	`loadData(sub)` → set `data`, `locked`, `ready`; local mode also sets `sync='local'` and calls `navigator.storage.persist()`.
Keep-alive pull effect	`session.sub` or `ready` changes	Google sessions: `pull()` once at launch (may renew the token).
`onVis` **(visibilitychange)**	tab hidden / shown	Hidden ≥ 60 s with a PIN → `setLocked(true)`. Shown: *only if a valid token exists*, push if dirty, otherwise `pull()` (throttled to once per 15 s).

Handler / effect	Trigger	Behaviour
`onOnline`	`online` event	`pull()`, again only with a valid token.
`schedulePush(ms=1500)`	after every commit	Debounces pushes so a burst of edits becomes one Drive write.
`push()` / `pull()`	see chapter 7	The two halves of sync.
`classify(error)`	any sync failure	Maps an error to `offline` (no network / TypeError), `auth` (401/403 or popup/auth messages) or `error`.

6.6 Other contexts and hooks

- `DialogProvider` / `useDialog()` (`components/dialog.jsx`): `await confirm({title, message, confirmText, danger})`, `await alert({...})`, `await choose({ actions:[{label,value,variant}] })`. One dialog at a time, Escape cancels (resolves `null`). These replace `window.confirm`, which looks out of place in an installed app and cannot offer three choices.
- `useInstall()` (`lib/install.js`): exposes `canPrompt`, `ios`, `installed` and `prompt()`.
- `useGroup(id, role, hint)` (`lib/useGroup.js`): see chapter 10.

> **WATCH OUT**
>
> **A real bug we hit:** after introducing `useDialog`, one button still called `confirm('Log out?')` with a plain string. The function returns a Promise, which is always truthy, so the button logged the user out immediately without asking. Always `await` the dialog helpers.

7 · Persistence and Drive sync

7.1 On-device storage: `idb.js`

A 60-line wrapper turns IndexedDB into three promise functions: `idbGet(key)`, `idbSet(key, value)`, `idbDel(key)`. One database (`pocketzen`), one object store (`kv`). If IndexedDB is unavailable (some private modes) it silently falls back to `localStorage` under `pz:kv:<key>`.

IndexedDB key	Content
`data:<sub>`	The user's document (`sub = 'local'` for no-account users).
`group:<fileId>`	Last known state of a shared group (works offline).
`outbox:<fileId>`	The admin's group edits not yet saved to Drive.

> **WHY THIS WAY**
>
> **Why IndexedDB and not localStorage?** localStorage is synchronous, limited to roughly 5 MB, and stores only strings. IndexedDB is asynchronous, has a far larger quota and is the better target for the browser's "persistent storage" request. Small flags still live in localStorage because they must be readable synchronously at startup (session, theme, token).

> **NOTE**
>
> **Migration.** Early versions kept the document in `localStorage['pz:data:<sub>']`. `loadData()` in `store.jsx` moves it into IndexedDB the first time it finds one, then deletes the old copy.

7.2 Cloud storage: the Drive app-data file

For Google users the same document is stored as `pocketzen-data.json` in the **app-data folder** of their Drive. That folder is hidden, private to this OAuth client and invisible in the normal Drive UI (`drive.appdata` scope). `google.js` exposes two calls:

- `driveLoad(sub, hint)` — find the file (id cached in `pz:fileId:<sub>`), download JSON; `null` if none exists.
- `driveSave(sub, data, hint)` — update the file with `PATCH …?uploadType=media`, or create it with a `multipart` upload the first time.

7.3 Dirty and synced markers

Key (localStorage)	Meaning
`pz:dirty:<sub>`	Set on every commit; cleared after a successful push if nothing changed in the meantime. "This device has changes Drive hasn't seen."

Key (localStorage)	Meaning
`pz:synced:<sub>`	The Drive file's `updatedAt` that this device last saw or wrote. Used to detect that *someone else* wrote to Drive since.

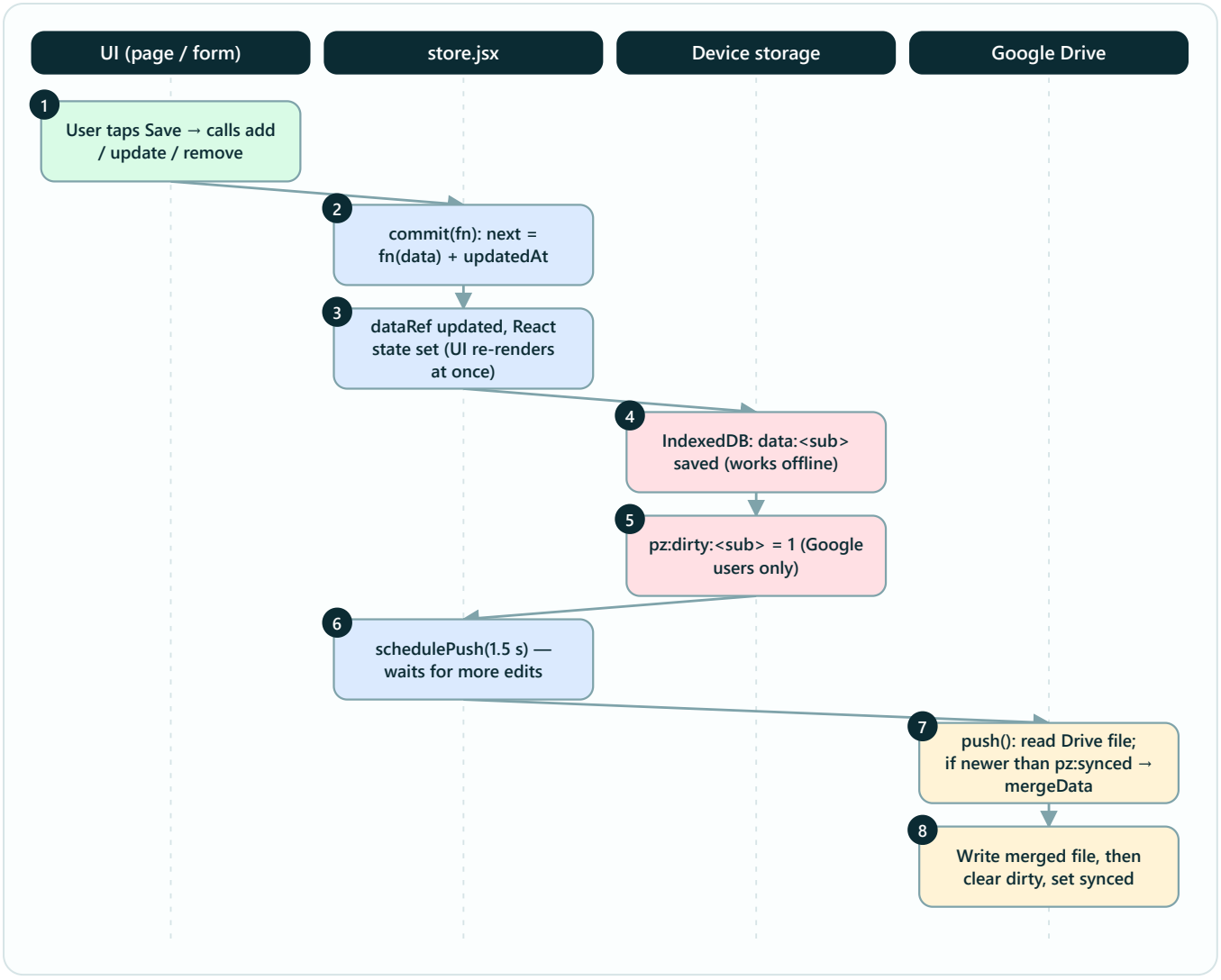
7.4 The two halves of sync

`pull()` — run at launch and when the app returns to the foreground

1. Load the Drive document (`remote`).
2. If `remote.updatedAt > local.updatedAt` and this device is **not dirty** → adopt the remote document (persist to IndexedDB, set `synced`).
3. Else if there is no remote, or the device is dirty, or local is newer → call `push()` .
4. Else they are identical → just record `synced` .

`push()` — runs after edits (debounced)

1. Re-read the Drive file. If its `updatedAt` is newer than `pz:synced` , **another device wrote since we last synced**: compute `merged = mergeData(remote, local)` (settings and PIN preferring local), replace local state with it.
2. `driveSave` the (possibly merged) document and record `synced = its updatedAt` .
3. If the user edited again while the upload ran, keep `dirty` and schedule another push; otherwise clear it.



WHY THIS WAY

The bug this design fixes. The first version simply uploaded the local document. With two devices open, the one holding an *older* copy overwrote the other's newer changes (a record added on the phone vanished when the laptop saved). It surfaced to the user as "data deleted on refresh". The reproducing test used a fake Drive shared by two browser contexts. The fix is the read-merge-write in step 1 above.

7.5 `mergeData(base, inc)` — combine without duplicates

The same function is used by the push merge, by "Connect Google Drive" when Drive already has data, and by importing a backup in "Merge" mode.

Collection	Rule
accounts	Matched by id *or* case-insensitive name; the incoming duplicate is dropped and its id remapped.
categories	Matched by id or (type + name). So two "Salary" income categories from two fresh installs become one.
transactions	Dedupe by id; `accountId / toAccountId / categoryId` remapped to the kept ids.

Collection	Rule
budgets	One per category.
emis, reminders	Union by id (EMIs' account remapped).
people, loans	People matched by name; loans union by id with `personId` and event `accountId` remapped.
groups	Union by id.
settings, pin	Base settings; optional-page flags OR-ed; PIN from whichever side has one.

> **WATCH OUT**
>
> **Deliberate trade-off:** a merge is a *union*, so it never loses data but cannot know about deletions. If device A deletes a record and device B (stale) edits something, the record can come back. We chose "never silently lose money records" over "never resurrect a deleted one". Real per-record tombstones would fix it at the cost of more code; the group engine (chapter 10) does use per-entity timestamps because its data is smaller and more structured.

7.6 Offline behaviour

- Reads and writes always hit IndexedDB, so the app is fully usable offline.
- While offline, `push()` fails; the status becomes `offline`, `dirty` stays set.
- When the device is back online (and a token is valid) or the user taps Reconnect, the next push merges and uploads.
- The service worker precaches the app shell, so the page itself loads without a network.

7.7 Switching between local and Google (`CloudCard.jsx`)

1. Offer a backup file first (optional).
2. Sign in, then `driveLoad`.
3. If Drive has user data, ask: **Merge both** (default, `mergeData(remote, local)`) · **Keep Drive data only** · **Replace Drive with this device** · Cancel.
4. `switchToGoogle(profile, next)` writes `next` under the Google `sub`, marks it dirty and removes the local copy. The normal push then uploads it.

The reverse, `disconnectGoogle()`, keeps a complete copy on the device and never touches the Drive file.

8 · Authentication and security

8.1 What "login" means here

PocketZen has **no user accounts of its own**. "Logging in" means getting permission from Google to read and write files in the user's Drive, using **Google Identity Services (GIS), token model**. The browser asks Google for an *access token*; the token is then sent as `Authorization: Bearer …` on Drive REST calls made directly from the browser.

> **WHY THIS WAY**
>
> **Why the token model and not "Sign in with Google" ID tokens or the code flow?** An ID token proves who you are but gives no Drive access. The authorization-code flow gives long-lived refresh tokens, but needs a server to keep a client secret, and we have no server. The token model needs no secret and no server. The price is that tokens last only about an hour and cannot be refreshed silently in the background without a window (see 8.4).

8.2 Scopes (permissions)

Scope	When requested	Gives access to
`openid email profile`	Every sign-in	Name, email, picture — shown in Profile and attached to group membership.
`…/auth/drive.appdata`	Every sign-in	The hidden app-data folder: `pocketzen-data.json`.
`…/auth/drive.file`	Only when the user creates their first group (`enableGroupScope()`)	Only files this app created — the group files. Not the rest of Drive.

All three are classed by Google as non-sensitive, so the OAuth app can be published without a security review. `drive.file` is requested *incrementally*: `enableGroupScope()` sets `pz:groupScope = '1'` and requests a token; from then on `scopes()` includes it. A second device shows an "Allow" banner the first time that admin opens a group there.

> **ADVANTAGE**
>
> Most users only ever see the basic permission. The extra Drive permission appears only for people who create a group, at the moment it makes sense.

8.3 The login flow

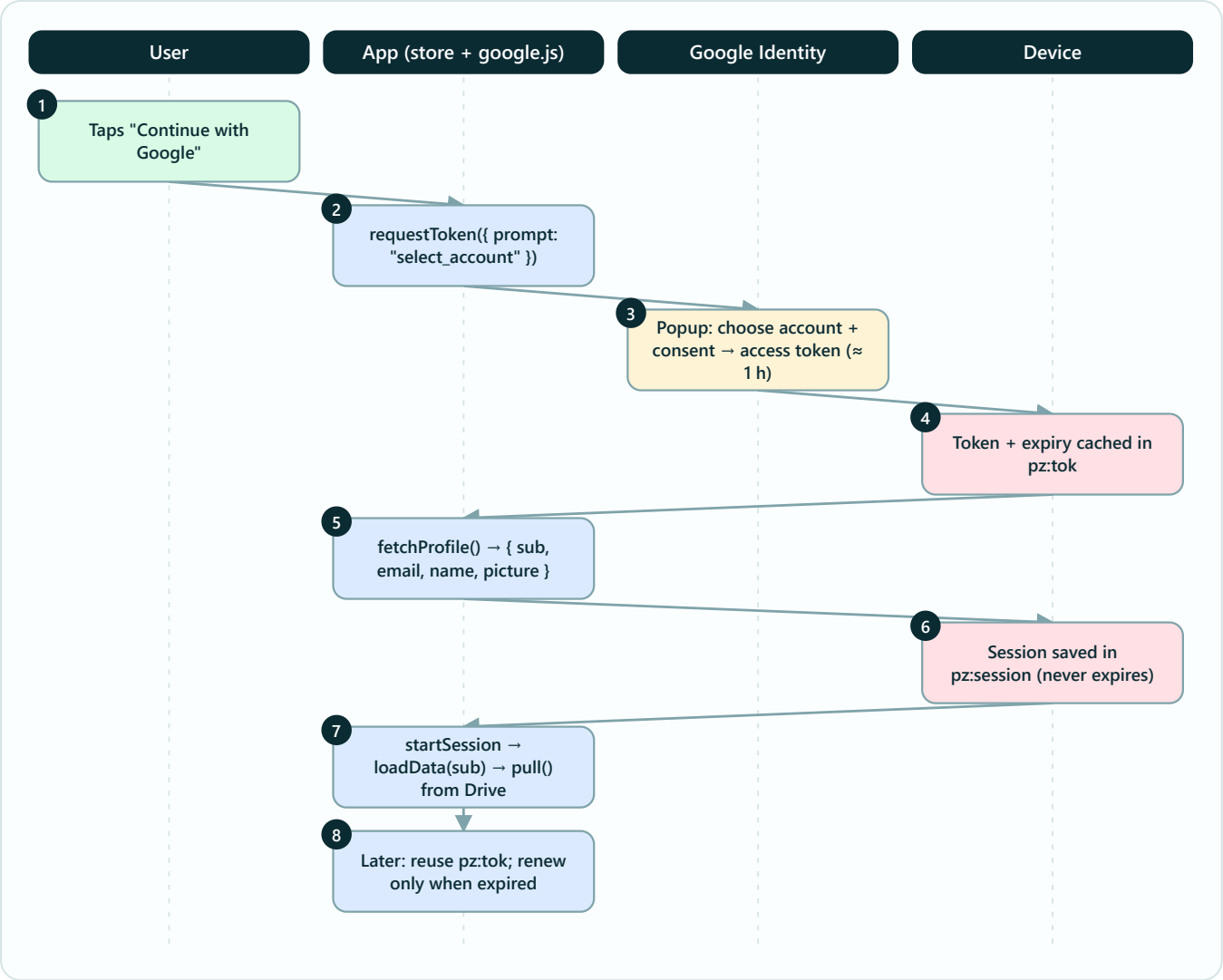


Figure 8.1 — First Google sign-in and later token reuse.

```
// google.js
export async function requestToken({ prompt = '', hint, scope } = {}) {
  const oauth2 = await waitForGis()            // the GIS script from index.html
  return new Promise((resolve, reject) => {
    oauth2.initTokenClient({
      client_id: CLIENT_ID, scope: scope || scopes(), hint, prompt,
      callback: (resp) => { token = resp.access_token; expiresAt = now + resp.expires_in*1000
                            saveToken(); resolve(token) },
      error_callback: (err) => reject(new Error(err?.type)),
    }).requestAccessToken()                       // opens Google's window
  })
}
```

- `login()` uses `prompt: 'select_account'` (shows the account chooser) → `fetchProfile()` → `startSession({ ...profile, mode:'google' })`.
- The session (`pz:session`) has **no expiry**. It lasts until the user logs out, which satisfies "keep me signed in".
- `loginLocal()` creates `{ sub:'local', mode:'local' }` and skips Google entirely.

8.4 Tokens: caching and renewal

Because there is no refresh token, every hour the app must ask Google again. That request opens a (usually auto-closing) window. Two rules keep this unobtrusive:

1. **Cache the token**
   The token and its expiry are stored in `localStorage['pz:tok']`. At module load, a token valid for more than one more minute is restored. Reloading, refreshing or reopening the installed app within the hour therefore reuses it: **no window**. The cache is cleared on logout and on a 401.

2. **Background work never opens the window**
   `hasLiveToken()` is checked before any timer, focus or "back online" sync. Only **launch**, a **user action** (saving an edit, tapping Reconnect, opening a group) or an explicit button may request a new token.

> **WHY THIS WAY**
>
> **What users saw before:** the token lived only in memory, so every page load opened Google's sign-in window. A test counting `requestAccessToken()` calls now shows: first launch 1, three reloads 0, expired token 1, focus/online with an expired token 0, saving an edit 1.

> **WATCH OUT**
>
> A token in `localStorage` is readable by any script on the same origin, so an XSS bug could steal it for up to an hour. The scopes are narrow (app data and files the app created) and the app contains no third-party scripts besides Google's, which is the usual accepted trade-off for a browser-only OAuth app.

8.5 Calling Drive: the `api()` helper

```
async function api(url, opts, hint) {
  const t = await ensureToken(hint)                    // cached → else renew
  let res = await fetch(url, { ...opts, headers: { Authorization: `Bearer ${t}` } })
  if (res.status === 401) { forgetToken(); /* renew once, retry */ }
  if (!res.ok) throw Object.assign(new Error(`Google API ${res.status}`), { status: res.status })
  return res
}
```

8.6 The PIN lock

Piece	Detail
Storage	`data.pin = { salt, hash, recovery? }`; the PIN itself is never stored.
Hashing	`hashPin`: PBKDF2-SHA-256, 150 000 iterations, 16-byte random salt, 256-bit output (Web Crypto). `verifyPin` recomputes and compares.
When it locks	On launch if a PIN exists, and when the tab has been hidden for 60 seconds or more.
Brute force	Five wrong attempts → 30-second wait (`PinLock.jsx`). A 4-digit PIN is a casual-privacy lock, not strong security, and the UI does not claim otherwise.

Piece	Detail
Forgot PIN (Google)	`reverifyAccount(sub)` : sign in again with `select_account` and require the **same** `sub` ; then choose a new PIN. No email and no extra permission.
Forgot PIN (local)	A one-time **recovery code** (12 characters from a 32-letter alphabet without 0/O/1/I, shown as `XXXX-XXXX-XXXX`) is created when the PIN is set. Only its salted PBKDF2 hash is stored. Entering it allows a new PIN. If both are lost, the only way in is "erase data".

> **WHY THIS WAY**
>
> **History:** the first design emailed a 6-digit code through the Gmail API. That needs the sensitive `gmail.send` scope, which forces Google's verification process. Re-verifying with Google proves identity just as well and needs no extra scope, so the email flow was removed.

8.7 Encrypted backups

- Plain backup: JSON of the document **without the PIN** (`buildBackup`).
- Protected backup: `{ app, encrypted:true, v:1, salt, iv, data }` — AES-GCM, 256-bit key derived from the password with PBKDF2-SHA-256 (200 000 iterations), random 16-byte salt and 12-byte IV. A wrong password fails authentication (`bad_password`); a missing one raises `password_required` .
- `saveBackupFile` shares through the phone's share sheet when it accepts files, otherwise downloads. The file is typed `text/plain` because far more share targets accept it than `application/json` .

8.8 Public identifiers in the bundle

`VITE_GOOGLE_CLIENT_ID` and `VITE_GOOGLE_API_KEY` are compiled into the JavaScript. That is normal for a browser app and safe *if configured correctly*: the client id only works from the authorised origins, and the API key should be restricted to your website and the Drive API in Google Cloud. **No secret exists in this project.**

8.9 Threat model in one table

Risk	Mitigation / honest limit
Someone picks up an unlocked phone	Optional PIN, auto-lock after 60 s hidden.
Lost phone, no account mode	Data is gone unless backed up; the app nudges backups and offers encrypted files.
Someone gets a group link	They can *view* that group (by design). The admin can delete the group file to revoke.
XSS on our origin	No third-party scripts, no `dangerouslySetInnerHTML` , React escapes output. Residual: cached token (≤ 1 h).
Server breach	There is no server holding data.

9 · Feature modules

Each feature follows the same pattern: **a page** (UI) + **a pure module** (rules) + **data in the document** (changed only through `useApp()`).

9.1 Records and the Add sheet

- **Page:** `Records.jsx`. State: `mode` (day/week/month) and `cursor` (a Date). `calc.periodRange(mode, cursor)` returns inclusive `{from,to}` date strings; `summarize()` gives income / expense / net; `groupByDay()` builds the grouped list. Weeks start on Monday.
- **Form:** `AddSheet.jsx` handles income, expense and transfer in one modal. Transfers need two different accounts and no category.
- **Budget hint:** for an expense in a budgeted category it shows *left = budget − spent this month (excluding the record being edited)* and what remains after the typed amount (`calc.monthSpentByCategory`).
- Dates are `'YYYY-MM-DD'` strings and times `'HH:mm'`; sorting is plain string comparison.

9.2 Categories, Accounts, Budget

- **Categories:** add/edit/delete with emoji + colour. Deleting a used category is allowed: its records show as "Uncategorized" and its budget is removed. Tapping a category opens `CategoryDetail` (all its transactions, Monthly/Weekly/Daily/All filter, budget bar).
- **Accounts:** balance is computed (`accountBalance`), never stored. An account with records cannot be deleted (the dialog says why).
- **Budget:** one monthly limit per expense category; the page sums that month's expenses per category and shows progress (amber from 80 %, red over 100 %).

9.3 EMIs (`emi.js` , `Emi.jsx`)

```
paidCount  = paidBefore + payments.length
paidAmount = paidBefore × emiAmount + Σ payments.amount
pending    = max(0, totalMonths − paidCount)
remaining  = max(0, totalAmount − paidAmount)
nextDue    = addMonths(startDate, paidCount)       // month-end safe: Jan 31 → Feb 28
nextAmount = pending === 1 ? remaining : emiAmount // the last instalment clears rounding leftovers
```

"Pay EMI" records a payment and, if an account is chosen, also adds an expense under an auto-created **"EMI & Loans"** category (the payment keeps the `txId`, so "Undo last" removes both). Completed EMIs move to their own tab.

9.4 Loans and people (`loans.js` , `Loans.jsx`)

A **loan** belongs to a **person** and has a direction (`lent` = they owe you, `borrowed` = you owe them). Its entire state is a list of dated **events**; every number on screen is derived from them by `loanStats(loan, asOf)` .

Event kind	Effect
`principal`	Money given/taken (the first one is the opening amount; later ones are top-ups).
`payment`	`{ amount, principalPart, interestPart }` — the split is chosen by the user ("interest first" by default via `suggestSplit`).
`interest`	A manual interest charge.
`terms`	Interest rate / fixed amount changes from this date onward.

Interest type	How it accrues
none	—
fixed	The fixed amount is added on every monthly anniversary of the start date *while principal is outstanding*.
simple	`P × monthlyRate × (wholeMonths + days/30)` per segment, where `monthlyRate = rate/100` (per month) or `rate/100/12` (per year).
compound	Like simple, but each whole month uses `(P + unpaidInterest) × monthlyRate`; the part-month remainder is pro-rated.
manual	Nothing automatic; the user adds interest events.

The engine sorts events by date, walks them in order, and before applying each event "advances" accrual from the previous date to that date. Rate changes and repayments therefore split the timeline into segments. Outputs: principal in/paid/balance, interest charged/paid/due, `outstanding = principal + max(0, interestDue)`, `settled`, `nextInterestOn`. `personSummary()` adds a person's loans (receivable minus payable) and `buildStatement()` creates the WhatsApp-ready text.

> **WHY THIS WAY**
>
> **Why events instead of storing balances?** A history of events can be edited (delete a wrong payment) and every figure is recalculated correctly. Stored balances would need manual repair after any correction. Verified numbers: ₹10 000 at 2 %/month for 3 months → ₹600; 12 %/year on ₹12 000 for 6 months → ₹720; compound 2 %/month for 3 months → ₹612.08.

Optionally an entry is also recorded as a normal income/expense in an account (categories "Money lent", "Money borrowed", "Loan repayment received/paid"); the event stores the `txId` so deleting the entry removes that record too.

9.5 Reminders and notifications (`reminders.js`, `notify.js`)

- **Recurrence:** `occurrence(r, n)` returns the n-th due date (daily +n days, weekly +7n, monthly +n months clamped to month length). `nextOccurrence(r)` uses `doneCount` ("Mark done" increments it) and honours `until`.
- `checkDue(data)` (called every 30 s while the app runs) fires: due bill reminders; loan interest days (09:00, the loan's start day each month) and repay-by dates; and the daily expense nudge (skipped if a record already exists today). A "fired" map in `localStorage['pz:fired']` makes each reminder fire once.

- `fireReminder()`: if the app is visible → in-app toast + chime (Web Audio) + vibration; if hidden → a system notification through the service worker (`registration.showNotification`, so tapping it re-opens the app).

> **WATCH OUT**
>
> **Hard platform limits:** a website cannot schedule an alert for a phone that is locked or has the app fully closed (that needs a push server), and it cannot choose the system notification sound. The UI states this plainly; the sound/vibration toggle controls the in-app chime and whether the notification is "silent".

9.6 Backup, restore and the lite mode

- **Backup** (`BackupModal`): optional password; shares via the share sheet or downloads; records `settings.lastBackupAt`. Local-only users get a `BackupBanner` after 14 days (snoozable for 3).
- **Import:** reads the file, asks for a password if encrypted, then *Replace all* or *Merge* (`mergeData`).
- **Lite mode:** `App.jsx` filters the tab list to four pages, `User.jsx` hides notification/page settings, reminders don't run. Everything else is shared code, which is why upgrading to Google later is just a data move (7.7).

9.7 Profile page structure (`User.jsx`)

Sections in order: profile card → Backup & sync (`CloudCard`) → Appearance (theme, stored in `pz:theme`) → Preferences (currency, daily reminder) → Notifications → Pages (optional-page toggles) → Security (PIN) → Your data (export/import) → Share & feedback → Log out / Erase. Google-only sections are hidden in lite mode.

10 · Shared groups (split expenses)

10.1 The requirement and the design journey

Requirement: a group (trip, flat, event) with people, expenses and payments; suggested payments to settle up; a PDF summary; and a way to invite friends.

Attempt	Idea	Outcome
v1 — server	A Cloudflare Worker with one **Durable Object** per group, anyone with an id could join and edit.	Worked and was tested (25 concurrent edits, no lost update) — but it introduced a backend, which contradicted "no server" and brought hosting/privacy questions.
v2 — Drive-hosted (current)	The group is a **file in the admin's own Google Drive**. The admin edits. Everyone else gets a view-only link.	No server, no database. Matches the privacy promise. Trade-off: only the admin can change things.

> **WHY THIS WAY**
>
> The product owner chose simplicity: "no backend, joined users just view, admin has all the rights". That removed the hard part of collaboration (concurrent writers) and let us reuse Drive as storage. `wrangler.jsonc` still carries a migration that retires the old `GroupDO` class so the Cloudflare deploy stays clean.

10.2 Roles

Role	Who	Can	Stored as
admin	The Google user who created the group	Everything: add/edit/delete expenses, payments, people, events; rename; tick settle-up payments; invite; delete group	`data.groups[i].role = 'admin'`
viewer	Anyone with the link (no sign-in needed)	View, refresh, see who-pays-whom, download/share the PDF	Optional `role:'viewer'` entry if they save it (needs Google)

10.3 The group document (`groupState.js`)

```
{ id, name, currency, createdAt, updatedAt, version,
  members:  { [id]: { id, name, email, deleted, u } },
  events:   { [id]: { id, name, deleted, u } },
  expenses: { [id]: { id, title, amount, paidBy, split:[memberId], eventId, date, note, deleted, u } },
  payments: { [id]: { id, from, to, amount, date, note, settle, deleted, u } } }
```

Every entity has `u` (last-updated ms). Deleting sets `deleted:true` (a tombstone), so a deletion also wins over an older edit.

Operations and last-write-wins

Changes are small idempotent operations: `{ t:'set', c:'expenses', id, v:{…} }` or `{ t:'meta', v:{ name } }`. `applyOps(state, ops, now)` validates each one (`cleanEntity`: allowed fields only, id format, amount 0–1e9, date format, ≤ 2000 bytes, timestamps clamped to now + 60 s), ignores an op if the stored entity is newer, increments `version` when something changed, and enforces limits (60 members, 50 events, 3000 expenses, 3000 payments, 1 MB total). It never mutates its input. The same function is used when the admin saves and when the UI overlays unsaved edits on top of the last known state.

10.4 Money maths

- **Minor units.** `toMinor(x) = round(x × 100)`; all group arithmetic is on integers, so no floating-point drift.
- `splitEqual(total, ids)`: `base = ⌊total/n⌋`; the first `total − base×n` people get one extra cent, so the parts always sum to the total.
- `computeBalances`: per person `paid`, `share`, `sent`, `received`; `net = paid − share + sent − received` (positive = the group owes them). Net always sums to 0.
- `suggestSettlements`: repeatedly match the largest debtor with the largest creditor and transfer `min(debt, credit)` until all are zero. At most $n - 1$ payments, deterministic ordering.
- **Settle-up checkboxes:** ticking a suggestion creates a `payment` (`settle:true`); balances shrink and the suggestion disappears; unticking deletes that payment. The checkbox is the UI for a real payment record.

10.5 Lifecycle: create → share → view

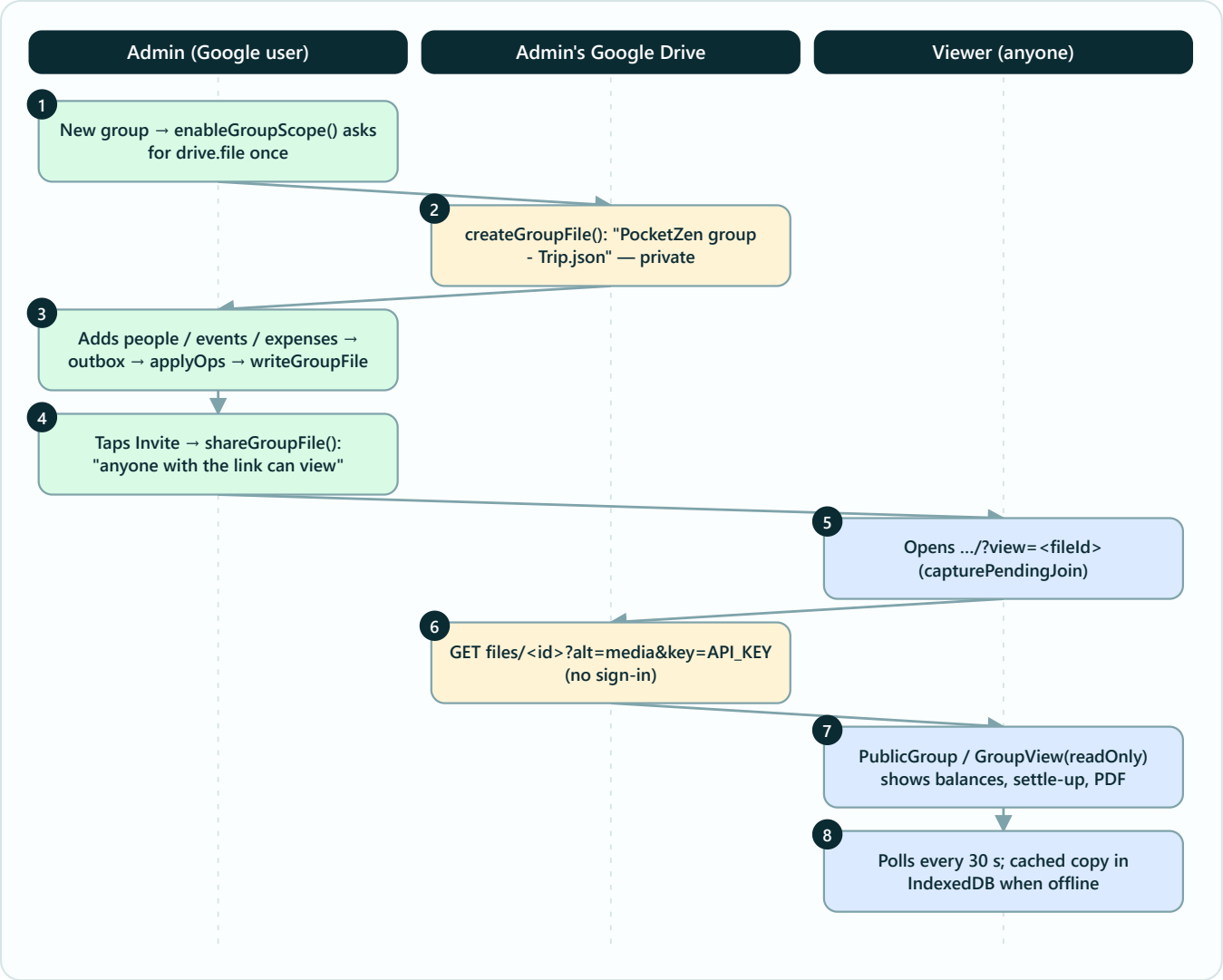


Figure 10.1 — Admin, Drive and viewer. Nothing in this picture is a PocketZen server.

1

Create

`enableGroupScope()` (first group only) → build a draft with the admin as the first member → `createGroupFile()` (multipart upload, **private**) → `writeGroupFile()` again so the real file id is stored inside → cache in IndexedDB → `add('groups', { id, name, role:'admin', me })`.

2

Edit (admin)

Forms call `dispatch(ops)` from `useGroup`. Ops go into an **outbox** (IndexedDB) and are applied optimistically on screen. The sync step re-reads the file, applies the outbox on top of what is in Drive (the admin may use several devices), and writes it back. Offline edits wait in the outbox.

3

Invite

Tapping **Invite** calls `shareGroupFile()` (`permissions.create` role reader, type anyone, `allowFileDiscovery:false`) and shows the link `…/?view=<fileId>`. Until then the file is private.

4 **View**

Opening the link: `capturePendingJoin()` stores the id; `App.jsx` shows `PublicGroup` (no session) which renders `GroupView` with a viewer entry. `readPublicGroupFile(id)` fetches `files/<id>?alt=media&key=API_KEY` — no token. Polls every 30 s; keeps a cached copy for offline.

5 **Save to my list (viewer with Google)**

Groups → Join accepts a link or id, reads the file, optionally lets the person pick "which one are you?" (to show their own balance) and stores a `viewer` entry.

6 **Delete**

`deleteGroupFile()` removes the file; viewers see "This group isn't available".

10.6 `useGroup(id, role, hint)`

Part	Behaviour
Cache	Loads `group:<id>` first, so the screen appears instantly and works offline.
Polling	Admin every 20 s, viewer every 30 s, plus on tab visibility and `online`. Admin polling needs a valid token; viewer polling needs none.
Outbox	Admin only. Unsent ops persist in `outbox:<id>` and survive reloads.
Shown state	`applyOps(lastServerState, outbox)` — what the admin typed appears immediately.
Status	`loading | live | offline | auth | scope | missing | nokey | badkey`. `scope`: this device has not been granted `drive.file` yet (shows an "Allow" button). `missing`: file deleted or no longer shared.

10.7 The sharing switch and the API key

Viewing by link needs a Google **API key** compiled into the app (`VITE_GOOGLE_API_KEY`). `groupSharingEnabled() = GROUP_SHARING && hasApiKey()`. When it is false: the Join and Invite buttons disappear, group links are ignored and groups remain a private, admin-only tool. Setting the key at build time turns sharing on with no code change; a constant in `src/config.js` can force it off. Error messages distinguish *no key*, *key rejected by Google* (wrong referrer restriction, Drive API not enabled) and *group not found*.

10.8 The PDF summary (`groupPdf.js`)

1. A `Doc` class draws A4 pages (794×1123 CSS px, rendered at 2×) on `<canvas>`: gradient header with logo, stat tiles, then tables (balances, suggested payments, payments recorded, every expense) with wrapping text, zebra rows, repeated table headers on a new page and "Page n of N" footers.
2. Each canvas becomes a JPEG (`canvas.toBlob`, quality 0.92).
3. `pdfFromJpegs()` writes a minimal PDF 1.4: catalog, pages, and per page a page object, an image XObject (`/DCTDecode`) and a content stream drawing it full-page; then the xref table and trailer.
4. `shareOrDownloadPdf()` uses the share sheet when it accepts files, else downloads.

WHY THIS WAY

Why canvas and not a PDF library? Standard PDF fonts cannot show "₹" or Tamil/Hindi/Chinese names. The browser can, so we let it render the text and embed the picture. Trade-off: the text in the PDF is not selectable. Tested with 90 expenses → 4 valid pages with mixed scripts.

11 · The UI layer

11.1 Styling system (`styles.css`)

- **Design tokens** as CSS custom properties on `:root`, taken from the logo's mint-to-teal gradient: `--g1 #8fe69a`, `--g2 #1ccbb0`, `--g3 #03a7c3`, `--grad`, plus `--ink`, `--muted`, `--surface`, `--line`, `--income`, `--expense`.
- **Dark mode**: `:root[data-theme='dark']` overrides the tokens. The attribute is set before paint by the inline script in `index.html`, and changed by `applyTheme()` in `User.jsx` (light / dark / auto follows the OS).
- **No component library**: a small vocabulary of classes — `.card`, `.list` / `.item`, `.btn` (`.primary` `.ghost` `.danger` `.sm`), `.chip`, `.seg` (segmented control), `.field` / `.input`, `.hero`, `.banner`, `.bar`, `.toggle`.
- **Sizes** were tuned for real screens: 14 px base text, 34 px icon buttons, 52 px floating button, compact lists. A visual pass on 390×844 and 1440×900 screenshots caught overlaps and oversized controls.
- **Text selection is disabled** app-wide (`user-select: none`) for an app-like feel; inputs, textareas, selects and elements with `.selectable` (recovery code, group id) stay selectable.

11.2 Responsive layout

Width	Navigation	Content
< 820 px (phones)	Fixed **bottom bar** (scrolls sideways when more than six tabs); floating add button above it	Single column, 14 px padding, safe-area insets for notches
≥ 820 px (tablet, laptop)	Left **sidebar** rail, 76 px, icons + small labels	Centered page, max 1080 px
≥ 1280 px (monitor)	Sidebar widens to 200 px with full labels and the sync status text	Content narrows to max 980 px so lines stay readable

The same `tabs` array renders both navigations, so they can never disagree.

11.3 Modals and dialogs

- `Modal` (`ui.jsx`): bottom sheet on phones, centred card on desktop. A module-level *stack* ensures that when several modals are open (person → loan → payment form) only the topmost reacts to Escape, and body scrolling stays locked until the last one closes. The callback is kept in a ref so the effect registers once and ordering never gets scrambled by re-renders.
- **Stacking order** is simply DOM order. A page that opens a form *from inside* another modal must render that form **after** the parent modal. (A real bug: "Add loan" opened behind the person screen until the render order was swapped.)
- **Dialogs** (`dialog.jsx`) are promise-based and sit above modals (z-index 120).

11.4 Patterns used across pages

Pattern	Where
Form modal with Save in the footer	every create/edit flow
Segmented control for modes	Records periods, Add sheet type, interest period
Chip groups for single/multi select	accounts, categories, group payer/split, interest type
"Hero" summary card with gradient	Records, Accounts, Budget, EMIs, Loans, Groups
Empty state with a call to action	every list page
Optimistic updates	all writes (state changes first, storage and Drive follow)
Inline validation messages (`role="alert"`)	forms; errors are explained in plain words, not codes

11.5 Accessibility basics included

- Icon-only buttons have `aria-label`; toggles are `role="switch"` with `aria-checked`; modals have `role="dialog"` and `aria-modal`; dialogs use `role="alertdialog"`.
- Visible focus rings (`:focus-visible`), `prefers-reduced-motion` disables animations, colour is never the only signal (labels like "owes/gets/over by").
- The PIN pad also accepts the physical keyboard.

11.6 Adding a page — the recipe

1. Create `src/pages/MyPage.jsx`; read data with `useApp()`, mutate with `add/update/remove`.
2. Put calculations in a pure `src/lib/mything.js` and unit-test them.
3. Add the new collection to `emptyData()`, to the array list in `normalize()` (`store.jsx`), and to `mergeData` (`merge.js`) — otherwise import/merge/sync will silently drop it.
4. Add a row to `TABS` in `App.jsx` (with a settings flag if it is optional) and a toggle in `User.jsx`; add it to `LITE_TABS` only if lite users should have it.

12 · PWA, offline and notifications

12.1 What makes it installable

Ingredient	Where
HTTPS	Cloudflare serves the site over HTTPS (required for service workers, install and notifications).
Web app manifest	Generated by `vite-plugin-pwa` from `vite.config.js`: name/short_name PocketZen, `display: standalone`, `start_url: '/'`, theme `#0fb5a8`, icons 192 / 512 / maskable 512.
Service worker	Generated by Workbox (`registerType: 'autoUpdate'`): precaches the built JS, CSS, HTML and images; new versions install silently.
Icons	`npm run icons` (`scripts/make-icons.mjs`, sharp) removes the logo's white background (alpha derived from darkness) and writes every size.

12.2 Service worker details

```
VitePWA({
  registerType: 'autoUpdate',
  workbox: {
    importScripts: ['sw-notify.js'],          // our own handlers, loaded into the generated worker
    globPatterns: ['**/*.{js,css,html,png,svg,ico}'],
    navigateFallback: '/index.html',          // SPA: any navigation serves the app shell …
    navigateFallbackDenylist: [/^\/privacy\.html$/, /^\/home\.html$/],  // … except these pages
  },
})
```

- `public/sw-notify.js` adds **`notificationclick`** (focus an open window or open the app) and a **`push`** handler (ready for a future push server; unused today).
- The denylist matters: without it the service worker would answer `/privacy.html` with the app instead of the policy page (the pages Google's consent screen links to).
- Drive and Google requests are cross-origin `fetch` calls and are never cached by the service worker.

> **WATCH OUT**
>
> **Updating:** because the app shell is precached, a user sees a new deploy on the *next* load after the new worker activates. If a fix "doesn't show", reload twice or clear site data.

12.3 The install prompt (`install.js`, `InstallBanner`)

- **Android / desktop Chrome:** a module-level listener stores the `beforeinstallprompt` event as soon as the page loads (before React mounts). The banner's **Install** button calls `event.prompt()`.

- **iPhone Safari** fires no such event, so the banner shows the manual steps (Share → Add to Home Screen).
- Hidden when already standalone (`display-mode: standalone` or `navigator.standalone`) and for 7 days after dismissal (`pz:installDismissed`). It also appears on the login page.

12.4 Notifications pipeline

Step	Code
Ask permission	`askNotificationPermission()` — never on page load (browsers block that). It is requested when the user enables reminders or taps Allow in `NotifyBanner` / the Profile card.
Decide when to fire	`checkDue()` every 30 s and when the tab becomes visible (chapter 9.5).
Foreground	Chime (Web Audio, three sine tones) + vibration + in-app toast. Audio is unlocked by the first tap (`unlockAudio` in `main.jsx`).
Background tab / installed app	`systemNotify()` → `navigator.serviceWorker.ready` → `registration.showNotification(title, { body, tag, vibrate, requireInteraction, data })` . If sound is off the notification is `silent: true` (the spec forbids combining `silent` and `vibrate`).
Tap on notification	`notificationclick` in `sw-notify.js` focuses or opens the app.

> **WHY THIS WAY**
>
> **Why no push server?** It would be the only server in the project and would hold users' reminder text. We accepted the limit that alerts depend on the app being able to run, and documented it. The service worker already contains the `push` handler, so adding a push backend later would be additive.

12.5 Offline summary

Situation	What happens
Open the app with no network	Service worker serves the cached shell; data loads from IndexedDB.
Edit while offline	Saved locally and marked dirty; syncs when a token and network exist.
Open a group while offline	Shows the cached group; admin edits queue in the outbox.
First visit with no network	Cannot load (nothing cached yet), as for any website.

13 · Build, configuration and deployment

13.1 Commands

Command	Does
`npm install`	Install dependencies.
`npm run dev`	Vite dev server at `http://localhost:5173` with hot reload.
`npm run build`	Production bundle into `dist/` plus the service worker and manifest.
`npm run preview`	Serves `dist/` so you can test the installable build.
`npm run icons`	Regenerate PWA icons from `src/assets/logo-source.jpg`.
`npm run poster -- <url> --form <formUrl>`	Render the share poster (needs a build first) and refresh `public/share-poster.jpg`.
`npm run docs`	Rebuild this PDF from `docs/architecture`.

13.2 Environment variables

Variable	Purpose
`VITE_GOOGLE_CLIENT_ID`	OAuth web client id (sign-in).
`VITE_GOOGLE_API_KEY`	Browser API key, restricted to the site and the Drive API. Enables group view links (optional).

> **WATCH OUT**
> Vite replaces `import.meta.env.VITE_*` with literal text **at build time**. On Cloudflare they must be **build** variables, and the site must be rebuilt after changing them. A runtime Worker variable has no effect on the bundle.

13.3 Google Cloud checklist

1. Create a project; enable the **Google Drive API**.
2. Google Auth Platform: name, support email, home page `/home.html`, privacy policy `/privacy.html`; Data Access scopes `openid, email, profile, drive.appdata, drive.file`; publish to Production (no review needed for these scopes).
3. OAuth client (Web): authorised JavaScript origins = `http://localhost:5173` + the production origin (no trailing slash).
4. API key: Websites restriction (`https://<domain>/*`, `http://localhost:5173/*`) and API restriction = Drive API.

> **NOTE**
>
> **Consent screen name.** Google shows the app name and logo only after *brand verification*, which needs a domain you own. On a shared `workers.dev` address the consent screen shows the address instead. A custom domain is the way to get "Sign in to PocketZen".

13.4 Cloudflare Workers (free)

```
// wrangler.jsonc
{
  "name": "pocketzen",
  "main": "worker/index.js",                // export default { fetch: (req, env) =>
env.ASSETS.fetch(req) }
  "compatibility_date": "2026-10-01",
  "assets": { "directory": "./dist", "binding": "ASSETS", "not_found_handling": "single-page-
application" },
  "migrations": [ { "tag": "v1", "new_sqlite_classes": ["GroupDO"] },
                  { "tag": "v2", "deleted_classes": ["GroupDO"] } ]   // retires the old server-based
groups
}
```

Cloudflare setting	Value
Git repository	ArunKathir98/PocketZen, branch `main` (every push redeploys)
Build command	`npm run build`
Deploy command	`npx wrangler deploy`
Build variables	`VITE_GOOGLE_CLIENT_ID`, `VITE_GOOGLE_API_KEY`, `NODE_VERSION=22`

> **WHY THIS WAY**
>
> **Why Workers static assets and not Pages?** Cloudflare is steering new projects to Workers; it is free for static sites, has global CDN, and the same config can later grow API routes. The tiny `worker/index.js` exists only so the migration history (which retires an old Durable Object) can be applied.

13.5 Static pages for Google's consent screen

`public/home.html` (what the app is) and `public/privacy.html` (privacy policy) are plain HTML files copied to the site root. Google requires both URLs to publish an OAuth app. They are deliberately not React: crawlers read them without running JavaScript.

13.6 Release checklist

1. `npm run build` succeeds locally.
2. Smoke test the preview build: sign in, add a record, reload, check the status pill says "Synced to Drive".
3. Push to `main`; watch the Cloudflare deploy log.
4. Open the live URL in a private window; reload twice (service-worker update).

5. If environment variables changed, confirm they are **build** variables and redeploy.

14 · Testing approach

The repository has no test runner configured yet. During development, correctness was checked with **throw-away scripts** in two layers. Describing them here so you can reproduce or formalise them.

14.1 Layer 1 — pure-logic tests (Node)

Because the domain modules are plain functions, they can be bundled (the project's own Rolldown, since the source uses extension-less imports) and exercised with `node:assert`.

Module	What was asserted
merge, backup, pin	Same-named accounts/categories unify; ids remapped; PIN excluded from backups; encrypted backup hides content, wrong password rejected; recovery code format and forgiving input.
loans	None/fixed/simple (monthly and yearly)/compound/manual; mid-way rate change; partial month (45 days); top-ups; repayments; fixed interest stops when principal is cleared; settled detection; person totals; statement text.
reminders + merge of loans	Interest-day and repay-by reminders fire once, not before 09:00, not for settled loans or when the page is off; people/loans merge with remapping.
groupState	Splits always sum to the total (also for 7 people, odd cents); balances sum to 0; settle-up ≤ n−1 transfers and clears everyone (80 random expenses); last-write-wins, idempotent replay, deletes, validation rejects bad input without mutating, limits, group-link parsing.

14.2 Layer 2 — end-to-end tests (Playwright + mocked Google)

A headless Chrome/Edge runs the production build. Network calls to Google are intercepted with `page.route`, and a small in-memory **fake Drive** implements exactly the rules the app depends on (owner's token for edits, public read only with a key and only when shared, app-data per user, delete). The Google script is replaced by a stub that counts `requestAccessToken()` calls.

Scenario	Checks
Core money flows	Optional pages hidden by default, enable in Profile, EMI add/pay/undo with linked expense, category detail, app dialogs, no native browser dialogs ever fired.
Notifications	Background reminder produces a real system notification (silent/vibrate options); foreground uses a toast; install banner flow.
Local mode	Reload persistence in IndexedDB, lite navigation, PIN → recovery code → forgot-PIN, encrypted backup + restore, connect to Google (empty Drive and merge), disconnect, erase.
Loans	Create, interest figures match the engine to the paisa, payments split, recorded-in-account with undo, statement share text, settled tab, persistence.

Scenario	Checks
Groups (Drive-hosted)	Admin permission request (`drive.file` only for groups), file private until Invite, public read-only page with no sign-in, no add/edit controls for viewers, PDF download, second admin device, offline outbox, delete, bad links, sharing off without a key.
Two-device sync	Two contexts, one account, one fake Drive: a stale device no longer overwrites the other's change.
Token cache	Sign-in windows: first launch 1, reloads 0, expired 1, background triggers 0, user edit 1.

14.3 What the tests caught (examples)

- **Data loss on refresh** — reproduced with two devices (7.4) before fixing.
- **A logout button without confirmation** — an un-awaited dialog promise.
- **A form opening behind another modal** — DOM-order stacking.
- **A sync pill stretching to full height** — a CSS class name (`.auth`) reused for two unrelated things.
- **New device can't open a group** — the "Allow" prompt was drawn on a screen that only appears once the group has loaded.

14.4 Recommended next step

> **TIP**
>
> Move these into the repository: add **Vitest** for the pure modules (`src/lib/*.js`) and **@playwright/test** for the flows, with the fake-Drive helper as a shared fixture, and run both in CI before deploy. The logic modules are already structured for this: no changes to production code are needed.

15 · Step-by-step implementation

This is the order the app was actually built in, written so you can follow it to build the same thing. Each step says **what** to do, **which files** are involved and **why it comes at that point**.

1

Scaffold, brand and PWA shell

Do: `npm i react react-dom lucide-react`, dev deps `vite @vitejs/plugin-react vite-plugin-pwa sharp`. Turn the logo into a transparent PNG and the icon set with `scripts/make-icons.mjs`. Define colour tokens in `styles.css` from the logo gradient; write the inline theme script in `index.html`. Configure the PWA plugin (manifest + Workbox).
Why first: a consistent theme and an installable shell make every later screen look right from day one.

2

Data model and local store

Do: write `defaults.js` (`emptyData()`), then `store.jsx` with `commit`, `add/update/remove`, persisted to `localStorage` at first (moved to IndexedDB in step 11). Add `util.js` (ids, dates, money) and `calc.js` (balances, periods).
Why: every page depends on the document shape. Fix it early and keep it a single JSON object.

3

Core pages

Do: Records + `AddSheet`, Categories, Accounts, Budget, Reminders, Profile; shared `Modal` / `Toggle` / `Empty` in `ui.jsx`; bottom-nav/sidebar in `App.jsx`.
Why: proves the store API (`add/update/remove`) is enough for real screens before adding networking.

4

Google sign-in and Drive sync

Do: create the Google Cloud project and OAuth client; load GIS in `index.html`; write `google.js` (`requestToken`, `fetchProfile`, `driveLoad`, `driveSave`); add `login`, `pull`, `push`, `schedulePush`, dirty flag to the store; build `Login.jsx`.
Why now: the local-first store already works, so sync is "just" mirroring one document.

5

PIN lock and keep-signed-in

Do: `pin.js` (PBKDF2), `PinPad`, `PinLock`, auto-lock on hidden ≥ 60 s; session never expires. First "Forgot PIN" emailed a code through Gmail, later replaced by Google re-verification (chapter 8.6).
Why: it needs both the store and auth, so it comes after step 4.

6

OAuth publication requirements

Do: add `public/home.html` and `privacy.html`, branding, remove sensitive scopes so no verification is needed.
Lesson: request the smallest scopes you can. One sensitive scope would have blocked publishing.

7 **Polish pass for real screens**

Do: shrink type/controls, fix overlaps, screenshot at phone and desktop sizes; replace browser `confirm` with `DialogProvider`.
Why: a first design is usually too big on phones; checking screenshots catches layout bugs a linter cannot.

8 **Optional pages: EMIs, category details**

Do: `emi.js` + `Emi.jsx`; `CategoryDetail`; feature flags (`showEmi`, `showReminders`) with toggles in Profile; `TABS` gets a 5th "flag" element.
Why flags: a simple app stays simple; power features are opt-in.

9 **Notifications, install prompt, sharing**

Do: `notify.js`, `sw-notify.js`, `install.js`, banners, `NotificationsCard`; share card with WhatsApp text and poster; feedback form link in `config.js`; poster generator.
Lesson: browsers can't ask for notification permission on load, can't pick a sound, and can't alert a locked phone; design the UI honestly around that.

10 **Local-only mode, backups, recovery code**

Do: `idb.js` and move the store to IndexedDB (with one-time migration); `loginLocal`; `backup.js` (AES-GCM) + `BackupModal`; recovery code in `pin.js`; `CloudCard` + `switchToGoogle` / `disconnectGoogle`; `merge.js`.
Why: having a clean "session mode" in the store made this additive rather than a rewrite.

11 **Loans & people**

Do: design the event model first, write `loans.js` and its tests (the maths must be right before any UI), then `Loans.jsx`, reminders for interest days, optional account recording, statements.
Why events: any correction stays consistent (chapter 9.4).

12 **Lite mode gating**

Do: `LITE_TABS` in `App.jsx`, hide Google-only settings in `User.jsx`, skip the reminder scheduler for local sessions.
Why: the product decision was "no account = simple tracker; Google = full app"; one set filter implements it.

13 **Groups: server prototype → Drive-hosted**

Do (final design): `groupState.js` (ops, LWW, splits, balances, settle-up), `useGroup.js` (cache, polling, outbox), group file functions in `google.js`, `Groups.jsx`, `GroupView.jsx`, `PublicGroup.jsx`, `groupPdf.js`, `pendingJoin.js`.
Lesson: the server version worked, but it contradicted the product's promise; choosing the simpler "admin edits, others view" model deleted the hardest problem (concurrent writers) and the server with it.

14 **Hardening from real-use bug reports**

Do: (a) read-merge-write on push + re-check Drive on foreground → no more overwritten data; (b) cache the token + forbid background sign-in windows; (c) sharing switch + clear API-key error messages; (d) remove unused code; (e) update README, privacy policy, home page, poster.
Lesson: every item here was found by a user or by a reproducing test — write the failing test *first*, then fix.

15.1 "If I had to rebuild it in a weekend" checklist

1. Vite + React + PWA plugin; tokens and theme.
2. `emptyData()`, `store.jsx` with `commit` and IndexedDB.
3. Records, Categories, Accounts pages with one reusable `Modal`.
4. Local-only session + backup file. **You already have a useful app here.**
5. Add Google token flow, `driveLoad/driveSave`, push/pull with dirty flag.
6. Add read-merge-write and the token cache *immediately* (don't repeat our bugs).
7. PIN lock; notifications; optional pages one at a time, each with a pure module and tests.
8. Groups last, as Drive files with an admin and view-only links.

16 · Decisions, advantages and trade-offs

Every row is a decision someone reviewing the code may question. Read the right-hand column before changing it.

Decision	Why	Advantage	Trade-off / when to revisit
No backend	Privacy promise, zero hosting cost, nothing to secure or maintain	No server breach surface, no accounts to manage, free forever	No refresh tokens, no push, no real-time collaboration. Revisit only if those become core.
One JSON document per user	Small data; simple to sync, export, merge	One Drive file, trivial backups, easy mental model	Whole file is rewritten on each sync; revisit at tens of thousands of records.
Local-first (IndexedDB)	Instant UI, offline use	Network failures never block the user	Needs a merge strategy when several devices write.
Context + refs, no Redux	One document, one write path	Almost no boilerplate; `commit()` is the single place to audit	Whole tree re-renders on any change; fine at this size, memoise if lists grow.
Pure logic modules	Money maths must be correct and testable	Bugs in numbers are fixed without opening React; unit tests are plain Node	Discipline needed: don't import React/DOM into `lib/*.js` logic files.
Event-based loans	Corrections must stay consistent	Delete a wrong payment and all figures recompute	Interest uses a day/30 approximation for part months and approximates compound after mid-month payments.
Integer minor units in groups	Equal splits must add up exactly	No floating-point drift, deterministic settle-up	Assumes 2 decimal places (JPY etc. would need a currency table).
Union merge	Never lose financial records	Safe with stale devices	A deletion on one device can be resurrected by another (7.5).
Groups as Drive files, admin-only edits	Keep "no server"; avoid concurrent-writer problems	Simple, private-by-default, free	Friends can't add expenses; viewers see updates after ≤ 30 s; needs an API key for public reads.
Token model + cached token	Only option without a server	No secret, no backend; reloads don't open Google's window	A window can appear about once an hour; token sits in localStorage (≤ 1 h).
Canvas → JPEG → PDF	Any language and "₹" must render	No font embedding, no PDF dependency	PDF text not selectable; larger file than vector text.

Decision	Why	Advantage	Trade-off / when to revisit
Early-return routing	One screen with tabs	No router dependency, no server rewrite rules	No deep links to pages. Add a router only if deep links become a requirement.
Hand-written CSS + tokens	Small app, custom look	No framework to learn; dark mode is a token swap	Class naming needs care (a clash once broke the sync pill).
Optional pages behind flags	Keep the default app simple	Casual users never see EMIs/loans/groups	Each new collection must be added in defaults, normalise and merge (11.6).
Lite mode for no-account users	Some people won't use Google	Wider audience; clean upgrade path	Fewer features by design; users must back up themselves.

16.1 Known limitations (be honest with users)

- Reminders alert only while the app can run; there is no alert for a fully closed or locked phone (needs a push server).
- Notification sound cannot be customised from a website.
- Without a Google account, data lives only on that device: clearing browser data or losing the phone loses it unless backed up.
- Group editing is admin-only; group view links are readable by anyone who has them until the admin deletes the group.
- Branding on Google's consent screen needs a domain you own.
- The PIN is a casual privacy lock, not strong security.
- Interest uses a simple month/day calendar approximation; confirm against a lender's statement for legal disputes.

17 · Maintenance playbook

The project is in bug-fix mode: no major features are planned. This chapter is the "where do I look?" guide.

17.1 Symptom → where to look

Symptom	First suspects	How to check
Data missing after refresh / on another device	`store.jsx` `push/pull`, `merge.js`	DevTools → Application → IndexedDB `pocketzen/kv` → `data:<sub>`; localStorage `pz:dirty:*`, `pz:synced:*`; Network: calls to `googleapis.com`.
Google window keeps appearing	`google.js` (token cache, `hasLiveToken` guards)	`localStorage['pz:tok']` present with a future `exp`? Remove it to simulate expiry.
"Reconnect" banner	Expired token + blocked popup	Tap Reconnect (a user gesture lets the popup open). Check the OAuth origin list.
Sign-in error `origin_mismatch`	Google Cloud OAuth client	The browser URL must exactly match an Authorised JavaScript origin (scheme, host, port, no trailing slash).
"Access blocked" for other people	OAuth consent still in Testing	Add them as test users or publish the app.
Group says it needs an API key	Build variable missing	Set `VITE_GOOGLE_API_KEY` as a **build** variable, redeploy. Join/Invite stay hidden without it.
Group says Google rejected the key	Key restrictions / Drive API	Add the site to the key's website referrers; enable the Drive API.
New device can't open its own group	Missing `drive.file` grant on that device	Use the "Allow" banner (`enableGroupScope`).
Balance / settle-up looks wrong	`groupState.js` (`computeBalances`), `calc.js`	Reproduce with the numbers in a tiny Node script; the functions are pure.
Loan interest differs from a lender's figure	`loans.js` (day/30, compounding model)	Check start date, rate period, events order; compare per segment.
No notification	`notify.js`, permission, SW	Notification permission granted? App hidden (foreground uses a toast)? Service worker active? OS focus mode?
No install button	HTTPS/manifest/dismissal	Lighthouse PWA checks; `pz:installDismissed` (7-day snooze).
A fix "doesn't show" after deploy	Precached service worker	Reload twice, or Application → Service Workers → Unregister.

Symptom	First suspects	How to check
Layout glitch on a phone	`styles.css` breakpoints (820 / 1280 px), safe-area insets	DevTools device toolbar at 390×844; check `.bottom-nav`, `.page`.

17.2 Conventions to keep

- **All writes go through** `commit()` (via `add/update/remove/setSettings`). Never mutate `data` directly.
- **New collection checklist:** `emptyData()` → `normalize()` → `mergeData()` → `hasUserData()` if relevant → page → tab.
- **Dates** are `'YYYY-MM-DD'` strings; times `'HH:mm'`; timestamps are ms numbers (`updatedAt`, `u`).
- **Money** is a plain number in personal data and integer minor units inside group maths.
- **Always** `await` **dialog helpers** (`confirm`, `alert`, `choose`).
- **Effects that start something must clean up** (StrictMode runs them twice in development).
- **Modals opened from inside a modal** must be rendered after it (DOM order = stacking order).
- **Keep** `lib/*` **logic files free of React and DOM** so they stay testable.
- **Background code must not open Google windows**: check `google.hasLiveToken()` first.
- **Never add a secret** to the project; only public identifiers belong in `VITE_*`.

17.3 Debug cheat-sheet

```
// DevTools console
Object.keys(localStorage).filter(k => k.startsWith('pz:'))      // every flag the app uses
JSON.parse(localStorage.getItem('pz:session'))                  // who is signed in, which mode
localStorage.removeItem('pz:tok')                               // pretend the Google token expired
localStorage.removeItem('pz:dirty:' + sub)                      // pretend nothing is waiting to upload
indexedDB.databases()                                           // is the 'pocketzen' database there?
```

17.4 Running a local copy end to end

1. `cp .env.example .env`, fill the client id (and API key if you want group links).
2. Add `http://localhost:5173` to the OAuth client's origins (and the API key's referrers).
3. `npm install && npm run dev`.
4. Restart `npm run dev` after editing `.env`; Vite reads it only at start.

17.5 What to do next (if anything)

- Move the tests into the repo (Vitest + Playwright) and run them in CI.
- Buy a domain, finish Google brand verification, and show "Sign in to PocketZen" on the consent screen.
- If real-time reminders on closed phones become important, add a push backend; `sw-notify.js` already handles `push`.

Appendix A · Reference

A.1 Storage keys

Key	Where	Purpose
`pz:session`	localStorage	Current session `{ sub, mode, email, name, picture, since }`. Removed on logout/erase.
`pz:tok`	localStorage	Cached Google access token + expiry (≤ 1 h). Cleared on logout and on a 401.
`pz:groupScope`	localStorage	"`1`" once this device was granted `drive.file`; makes later token requests include it.
`pz:dirty:<sub>`	localStorage	Local changes not yet uploaded to Drive.
`pz:synced:<sub>`	localStorage	Drive document `updatedAt` last seen/written by this device.
`pz:fileId:<sub>`	localStorage	Cached id of `pocketzen-data.json`.
`pz:theme`	localStorage	`light | dark | system`.
`pz:tab`	localStorage	Last selected tab.
`pz:pendingJoin`	localStorage	Group id from a `?view=` link, until handled.
`pz:installDismissed`	localStorage	Time the install banner was dismissed (7-day snooze).
`pz:backupSnooze`	localStorage	Backup reminder snoozed until this time (3 days).
`pz:localTipSeen`	localStorage	Welcome tip for no-account users shown.
`pz:fired`	localStorage	Map of reminders already fired (keys like `r:id:n`, `loan:id:YYYY-MM`, `daily:YYYY-MM-DD`).
`pz:kv:<key>`	localStorage	Fallback when IndexedDB is unavailable.
`pz:data:<sub>`	localStorage	*Legacy* location of the document; migrated to IndexedDB on first load.
`data:<sub>`	IndexedDB	The user document.
`group:<id>`	IndexedDB	Cached group state.
`outbox:<id>`	IndexedDB	Admin's unsaved group operations.

A.2 Google calls made by the app

Call	Auth	Used for
`GET oauth2/v3/userinfo`	Bearer token	Profile after sign-in (`fetchProfile`); re-verify for Forgot PIN.
`GET drive/v3/files?spaces=appDataFolder&q=…`	Bearer token	Find `pocketzen-data.json` .
`GET drive/v3/files/{id}?alt=media`	Bearer token	Download the user document or (admin) a group file.
`POST upload/drive/v3/files?uploadType=multipart`	Bearer token	Create the user file (app-data) or a group file.
`PATCH upload/drive/v3/files/{id}?uploadType=media`	Bearer token	Update either file.
`POST drive/v3/files/{id}/permissions`	Bearer token	Make a group readable by link (Invite).
`DELETE drive/v3/files/{id}`	Bearer token	Delete a group.
`GET drive/v3/files/{id}?alt=media&key=API_KEY`	API key only	Public read of a shared group (viewers, no sign-in).

A.3 Event handlers and timers

Where	Trigger	Effect
`store.jsx`	`visibilitychange`	Auto-lock after 60 s hidden; foreground sync (valid token only, throttled 15 s).
`store.jsx`	`online`	Pull (valid token only).
`store.jsx`	debounce 1.5 s	Push after edits.
`App.jsx`	interval 30 s + `visibilitychange`	Reminder scheduler (`checkDue`).
`useGroup.js`	interval 20 s (admin) / 30 s (viewer), `visibilitychange` , `online`	Refresh group.
`ui.jsx` Modal	`keydown` Escape	Closes only the top modal.
`dialog.jsx`	`keydown` Escape (capture phase)	Dismisses the dialog first.
`PinPad.jsx`	`keydown`	Digits and Backspace.

Where	Trigger	Effect
`main.jsx`	`pointerdown` / `keydown` once	Unlock Web Audio.
`install.js`	`beforeinstallprompt`, `appinstalled`	Remember / forget the install prompt.
`sw-notify.js`	`notificationclick`, `push`	Focus/open the app; show pushed notification.
`PinLock.jsx`	5 wrong PINs	30-second countdown.

A.4 Limits and constants worth knowing

Constant	Value
Google access token lifetime	≈ 1 hour (cached; renewed only when needed)
Auto-lock after being hidden	60 seconds
Push debounce / foreground pull throttle	1.5 s / 15 s
Reminder check interval	30 s
Group polling (admin / viewer)	20 s / 30 s
PIN hash / backup key derivation	PBKDF2-SHA-256, 150 000 / 200 000 iterations; AES-GCM 256
Recovery code	12 characters, alphabet of 32 (no 0, O, 1, I), shown `XXXX-XXXX-XXXX`
Group limits	60 members · 50 events · 3000 expenses · 3000 payments · 1 MB state · 2000 bytes per entity
Backup reminder (no-account users)	after 14 days, snooze 3 days
Install banner snooze	7 days

A.5 Glossary

Term	Meaning
sub	Google's unique id for an account; `'local'` for the no-account mode. Used as the key for all per-user data.
GIS	Google Identity Services — the browser library that issues access tokens.
App-data folder	A hidden Drive folder only this OAuth app can read (`drive.appdata`).
Lite mode	No-account session with only Records, Categories, Accounts (+ Profile).
Dirty / synced	"Local has unsent changes" / "the Drive version this device last saw".

Term	Meaning
Outbox	Queue of edits waiting to be saved to Drive (groups).
LWW	Last-write-wins: when two edits conflict, the newer timestamp wins.
Tombstone	A record kept with `deleted:true` so the deletion itself can be synced.
Minor units	The smallest currency unit (cents/paise) as integers.
PWA / Workbox / precache	Installable web app / Google's service-worker toolkit / files stored at install time for offline use.
Optimistic update	Change the screen first, confirm with storage/Drive afterwards.

End of guide · PocketZen v1.0 · generated from `docs/architecture` with `npm run docs`.